Studio Nx

S1-07 샘플 라이브러리 API — 함수 설명 합본 (백엔드 177~192)

2026-10-16

목차

1. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/177_models_sample.md
2. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/178_models_tag.md
3. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/179_models_sample_tag.md
4. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/180_sample_audio_read_audio_info.md
5. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/181_sample_audio_compute_waveform.md
6. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/182_sample_audio_write_waveform.md
7. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/183_sample_audio_check_upload.md
8. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/184_sample_service_save_sample.md
9. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/185_api_samples_upload.md
10. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/186_api_samples_list.md
11. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/187_api_samples_file.md
12. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/188_api_samples_waveform.md
13. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/189_api_samples_update.md
14. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/190_api_samples_tags.md
15. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/191_api_samples_delete.md
16. ai-notes/backend(fastAPI_Python)/S1-07 - 샘플 라이브러리/192_api_tags_list.md

Backend(FastAPI/Python) 177 — `Sample` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/sample.py`
작업	S1-07 샘플 라이브러리: ORM 모델 컬럼 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	176 `initResetForm()` · 178 `Tag` 모델

1. 하는 일

MySQL의 `samples` 테이블(사용자가 올린 샘플 파일 한 건의 정보)을 파이썬 클래스 `Sample`로 옮겨 적습니다. 그러면 SQL 문 대신 파이썬 코드로 샘플을 읽고 쓸 수 있습니다.

SQL	파이썬 (이 클래스가 있으면)
`SELECT * FROM samples WHERE sample_id = 3`	`db.get(Sample, 3)`
`source` 칸 읽기	`sample.source`는 `SampleSource.UPLOAD`
`INSERT INTO samples (owner_id, …) VALUES (…)`	`db.add(Sample(owner_id=…, …))`

테이블은 이 클래스가 만들지 않습니다. 테이블은 `studionx/db/02_phase2_sequencer.sql`로 이미 만들어져 있고, 이 클래스는 그 모양을 그대로 따라 적은 것입니다. 파일 자체는 디스크에 있고(경로 규칙 `samples/{owner_id}/{uuid}.{확장자}`), 이 행은 그 정보와 파형 파일 경로만 가집니다.

2. 언제 쓰나

샘플을 다루는 S1-07의 모든 백엔드 코드가 이 클래스를 씁니다.

쓰는 곳	읽거나 쓰는 칸
S1-07 업로드 (FR-221 ~ 223)	`owner_id`, `name`, `original_filename`, `file_path`, `mime_type`, `file_size`를 넣고, 올릴 때 파일에서 읽은 `duration_ms`, `sample_rate`, `channels`, `waveform_path`를 채웁니다
S1-07 라이브러리 목록 (FR-224)	`owner_id`로 거르고 `name`으로 찾으며, `deleted_at`이 비어 있는 행만 보여 줍니다
S1-07 이름 변경, 삭제 (FR-225)	`name`을 고치고, 삭제할 때 `deleted_at`을 기록합니다(소프트 삭제)
S1-07 저장 용량 (FR-227)	올리기 전 `file_size`의 합으로 남은 용량을 확인합니다
뒤 작업 (S1-10 등)	`clips.sample_id`와 `tracks.frozen_sample_id`가 이 행의 `sample_id`를 가리킵니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
```

```
from enum import StrEnum
from typing import TYPE_CHECKING

from sqlalchemy import BigInteger, Enum, FetchedValue, ForeignKey, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, INTEGER, TINYINT
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음 – 순환을 피함)
    from app.models.tag import Tag
```

import	가져오는 것	쓰는 곳
`datetime`, `Decimal`	날짜 시각 타입, 소수 타입	시각 칸, `bpm`
`StrEnum`	값이 문자열인 열거형	`SampleSource`
`BigInteger`, `String`, `Numeric`	SQL의 `BIGINT`, `VARCHAR(n)`, `DECIMAL(6,2)`	숫자·글자·소수 칸
`Enum`	SQL의 `ENUM(…)`	`source`
`text`	SQL 문장을 그대로 넘깁니다	기본값 `CURRENT_TIMESTAMP(3)`
`FetchedValue`	"DB가 값을 바꾼다"는 표시	`updated_at`
`ForeignKey`	다른 테이블을 가리킴	`owner_id` → `users`
`INTEGER`, `TINYINT` (MySQL 전용)	부호 없는 정수 (`INT UNSIGNED`, `TINYINT UNSIGNED`)	`duration_ms`, `sample_rate`, `channels`
`DATETIME` (MySQL 전용)	밀리초까지 저장하는 `DATETIME(3)`	시각 칸
`TYPE_CHECKING`	실행 중에는 안 불러오고 타입 검사 때만 불러옴	`Tag` 순환 import 회피
`relationship`	FK를 파이썬 속성으로 오가게 함	`sample.tags`

클래스 (칸 선언)

```
class SampleSource(StrEnum):
    UPLOAD = "UPLOAD"
    SEPARATION = "SEPARATION"
    EXPORT = "EXPORT"
    RECORDING = "RECORDING"
    BOUNCE = "BOUNCE"
    RESAMPLE = "RESAMPLE"


class Sample(Base):
    __tablename__ = "samples"

    sample_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    owner_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"), index=True)

    name: Mapped[str] = mapped_column(String(200))
    original_filename: Mapped[str] = mapped_column(String(255))
    file_path: Mapped[str] = mapped_column(String(500))
    mime_type: Mapped[str] = mapped_column(String(100))
    file_size: Mapped[int] = mapped_column(BigInteger)

    duration_ms: Mapped[int | None] = mapped_column(INTEGER(unsigned=True))
    sample_rate: Mapped[int | None] = mapped_column(INTEGER(unsigned=True))
```

```
    channels: Mapped[int | None] = mapped_column(TINYINT(unsigned=True))
    bpm: Mapped[Decimal | None] = mapped_column(Numeric(6, 2))
    music_key: Mapped[str | None] = mapped_column(String(10))
    waveform_path: Mapped[str | None] = mapped_column(String(500))

    source: Mapped[SampleSource] = mapped_column(Enum(SampleSource), server_default=SampleSource.UPLOAD)
    created_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP
(3)"))
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))

    tags: Mapped[list["Tag"]] = relationship(secondary="sample_tags", back_populates="samples")
```

SQL과 한 줄씩 비교

SQL (02_phase2_sequencer.sql)	파이썬
`sample_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY`	`BigInteger`, `primary_key=True`, `autoincrement=True`
`owner_id BIGINT UNSIGNED NOT NULL` + `KEY idx_samples_owner`	`ForeignKey("users.user_id")`, `index=True`
`name VARCHAR(200) NOT NULL`	`Mapped[str]`, `String(200)`
`duration_ms INT UNSIGNED NULL`	`Mapped[int | None]`, `INTEGER(unsigned=True)`
`channels TINYINT UNSIGNED NULL`	`Mapped[int | None]`, `TINYINT(unsigned=True)`
`bpm DECIMAL(6,2) NULL`	`Mapped[Decimal | None]`, `Numeric(6, 2)`
`source ENUM(…) NOT NULL DEFAULT 'UPLOAD'`	`Enum(SampleSource)`, `server_default=SampleSource.UPLOAD`
`updated_at … ON UPDATE CURRENT_TIMESTAMP(3)`	`server_default=text(…)`, `server_onupdate=FetchedValue()`
`deleted_at DATETIME(3) NULL`	`Mapped[datetime | None]`, `DATETIME(fsp=3)`

문법 풀이

식	뜻
`INTEGER(unsigned=True)`	`INT UNSIGNED` 입니다. **SQLAlchemy의 기본 `Integer` 는 `unsigned` 인자를 받지 않아** 오류가 납니다. 그래서 MySQL 전용 타입을 씁니다
`Numeric(6, 2)`	`DECIMAL(6,2)` 입니다. 합계 6자리, 소수 2자리이며, 읽으면 파이썬 `Decimal` 이 됩니다
`Mapped[str | None]`	`None` 이 없으면 `NOT NULL`, 있으면 `NULL` 을 허용합니다
`server_default=…`	값을 넣지 않으면 **DB가** 채우는 기본값입니다
`relationship(secondary=…, back_populates=…)`	중간 표(`sample_tags`)를 거치는 다대다 관계입니다. `back_populates` 는 반대쪽 `Tag.samples` 와 짝을 이룹니다

4. 흐름 (샘플 하나를 올릴 때, FR-221 ~ 223)

1. 파일을 디스크에 `samples/{owner_id}/{uuid}.wav` 로 저장합니다.

2. 파일에서 `duration_ms`, `sample_rate`, `channels`를 읽고(180), 파형 미리보기 데이터를 만들어 `waveform_path`를 정합니다. 읽지 못하는 파일은 업로드 단계에서 415로 거부합니다.
3. `Sample(owner_id=…, name=…, original_filename=…, file_path=…, mime_type=…, file_size=…)`를 만들어 `db.add`하고 `db.flush`합니다.
4. DB가 `sample_id`, `source=UPLOAD`, `created_at`을 채웁니다. `db.refresh`로 다시 읽으면 `sample.source`는 `SampleSource.UPLOAD`입니다.

5. 왜 이렇게

선택	이유
부호 없는 정수는 MySQL 전용 타입으로	`Integer(unsigned=True)`는 `TypeError`로 거부됩니다(Dialect 기본 타입이 `unsigned`를 받지 않습니다). `db/`의 DDL과 타입을 정확히 맞추려면 `INTEGER(unsigned=True)`를 써야 합니다. 앞선 Phase 1 모델(track.py 등)도 `TINYINT(unsigned=True)`를 같은 방식으로 씁니다
올릴 때 읽는 칸은 `NULL` 허용	`duration_ms`, `sample_rate`, `channels`, `bpm`, `music_key`, `waveform_path`는 파일을 읽어야 채워집니다. DB에서는 비워 둘 수 있게 했지만, 읽지 못하는 파일은 업로드 단계에서 415로 거부하므로(180) 실제 업로드에서는 비어 있지 않습니다. `bpm`, `music_key`는 채우는 작업이 뒤에 있어 지금은 비어 있습니다
삭제는 `deleted_at`(소프트 삭제)	이미 만든 클립이 이 샘플을 가리킬 수 있어(`clips.sample_id`), 행을 지우면 클립이 끊깁니다. 그래서 지운 표시만 하고 목록에서 뺍니다 (FR-225)
태그만 관계로 연결	지금 필요한 것은 태그뿐입니다(FR-224·225). `owner`나 `clips` 같은 관계는 쓸 곳이 생길 때 넣습니다
`bpm`은 `Numeric`	DDL이 `DECIMAL(6,2)`이고, 반올림 오차 없이 정확한 소수를 쓰기 위함입니다 (`128.00`)

6. 시험 (`tests/test_sample_model.py` 새 파일, 5개)

실제 MySQL의 `samples` 테이블과 비교했습니다. 넣은 행은 시험이 끝나면 되돌리므로(rollback) DB에 남지 않습니다.

시험	확인한 것
칸이 DB와 같은지	칸 이름과 순서, `NULL` 허용 여부, `owner_id` → `users` FK가 실제 테이블과 같습니다
기본값과 비어 있는 칸	5칸만 넣고 저장하면 `source`는 `UPLOAD`, `created_at`은 채워지고, `deleted_at`과 metadata 6칸은 비어 있습니다
metadata 값 저장	`duration_ms=1500`, `sample_rate=48000`, `channels=2`, `bpm='128.00'`, `music_key='Am'`, `waveform_path`를 넣으면 그대로 읽힙니다
소프트 삭제	`deleted_at`을 기록해도 행은 남습니다
소유자 삭제 차단	FK에 `ON DELETE`가 없어서, 소유자 행을 지우려 하면 DB가 막습니다(`IntegrityError`)

전체 시험 871개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 178 — `Tag` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/tag.py` (`Tag` 클래스)
작업	S1-07 샘플 라이브러리: ORM 모델 컬럼 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	177 `Sample` 모델 · 179 `SampleTag` 모델

1. 하는 일

MySQL의 `tags` 테이블(샘플에 붙이는 태그)을 파이썬 클래스 `Tag`로 옮겨 적습니다. 태그는 분류(`category`)별로 이름이 정해지고, `is_system`은 기본으로 제공하는 태그인지 나타냅니다. 샘플과 태그는 `sample_tags` 표로 이어집니다(다대다).

tag_id	category	name	is_system
1	`GENRE`	`Lo-fi`	1
2	`INSTRUMENT`	`Kick`	1
7	`MOOD`	`Deep`	0

`category`	뜻
`GENRE`	장르입니다 (예: `Techno`, `Lo-fi`)
`INSTRUMENT`	악기입니다 (예: `Kick`, `Piano`)
`MOOD`	분위기입니다 (예: `Deep`, `Bright`)
`ETC` (DB 기본값)	그 밖입니다

2. 언제 쓰나

쓰는 곳	하는 일
S1-07 라이브러리 태그 필터 (FR-224)	`category`별로 태그 목록을 보여 주고, 고른 태그가 붙은 샘플만 거릅니다
S1-07 태그 추가 (FR-225)	새 태그를 만들거나(`is_system=0`) 기존 태그를 붙입니다. 같은 분류에서 같은 이름은 만들 수 없습니다
(기본 제공 태그)	`is_system=1`인 태그는 클릭으로 고를 수 있게 기본으로 넣어 둡니다

3. 코드

import (파일 맨 위, `SampleTag`와 함께 씁니다)

```
from datetime import datetime
from enum import StrEnum
from typing import TYPE_CHECKING
```

```
from sqlalchemy import BigInteger, Boolean, Enum, ForeignKey, Integer, String, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base
```

import	가져오는 것	쓰는 곳
`StrEnum`	값이 문자열인 열거형	`TagCategory`
`Integer`, `String`	SQL의 `INT`, `VARCHAR(n)`	`tag_id`, `name`
`Boolean`	SQL의 `TINYINT(1)`	`is_system`
`Enum`	SQL의 `ENUM(…)`	`category`
`text`	SQL 문장을 그대로 넘깁니다	기본값 `1`, `CURRENT_TIMESTAMP(3)`
`DATETIME` (MySQL 전용)	`DATETIME(3)`	`created_at`
`relationship`	FK를 파이썬 속성으로 오가게 함	`tag.samples`

클래스 (칸 선언)

```
class TagCategory(StrEnum):
    GENRE = "GENRE"
    INSTRUMENT = "INSTRUMENT"
    MOOD = "MOOD"
    ETC = "ETC"


class Tag(Base):
    __tablename__ = "tags"

    tag_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(50))
    category: Mapped[TagCategory] = mapped_column(Enum(TagCategory), server_default=TagCategory.ETC)
    is_system: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
    created_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP
(3)"))

    samples: Mapped[list["Sample"]] = relationship(secondary="sample_tags", back_populates="tags")
```

SQL과 한 줄씩 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`tag_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY`	`Integer`, `primary_key=True`, `autoincrement=True`
`name VARCHAR(50) NOT NULL` + `UNIQUE KEY uq_tags_category_name (category, name)`	`Mapped[str]`, `String(50)`, `UniqueConstraint`는 표에 있음(칸이 아님)
`category ENUM('GENRE','INSTRUMENT','MOOD','ETC') NOT NULL DEFAULT 'ETC'`	`Enum(TagCategory)`, `server_default=TagCategory.ETC`
`is_system TINYINT(1) NOT NULL DEFAULT 1`	`Mapped[bool]`, `Boolean`, `server_default=text("1")`
`created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`	`server_default=text("CURRENT_TIMESTAMP(3)")`

문법 풀이

식	뜻
`Boolean` + `server_default=text("1")`	`TINYINT(1)`을 파이썬 `bool`로 다룹니다. DB가 `1`을 채우면 `True`로 읽힙니다
`Enum(TagCategory)`	정해진 값(`GENRE` 등)만 쓰게 합니다. `TagCategory.GENRE == "GENRE"`가 참입니다
`UNIQUE(category, name)`	두 칸을 **합쳐서** 유일합니다. 같은 분류에서 같은 이름은 한 번만, 다른 분류라면 같은 이름도 됩니다 (`MOOD`의 `Deep`과 `GENRE`의 `Deep`)
`relationship(secondary=…)`	`sample_tags` 표를 거치는 다대다입니다. 반대쪽은 `Sample.tags`입니다

4. 흐름 (태그를 붙일 때, FR-225)

1. 태그를 찾습니다. 없으면 `Tag(name="Deep", category=TagCategory.MOOD, is_system=0)`을 만들어 `db.flush`합니다.
2. `SampleTag(sample_id=…, tag_id=…)`로 이어 줍니다(179번).
3. 같은 분류에서 같은 이름을 또 만들면 DB가 막습니다(`UNIQUE(category, name)`).

5. 왜 이렇게

선택	이유
`is_system`을 파이썬 `bool`로	화면에서 "기본 태그인지"만 보면 되므로 `1/0` 대신 `True/False`가 읽기 쉽습니다
`category` 기본값을 `ETC`로	DDL의 기본값과 맞췄습니다. 태그는 보통 분류를 적어 만들지만, 적지 않으면 `ETC`가 됩니다
`tag_id`는 부호 없는 `INT`	`BIGINT`가 아니라 `INT`입니다(DDL 그대로). 태그는 샘플보다 적게 쌓이므로 `Integer`로 충분합니다
`created_at`만 있음	태그는 이름과 분류를 거의 고치지 않으므로 `updated_at`이 없습니다 (DDL 그대로)

6. 시험 (`tests/test_tag_model.py` 중 `Tag` 부분, 4개)

실제 MySQL의 `tags` 테이블과 비교했습니다. 넣은 행은 시험이 끝나면 되돌립니다.

시험	확인한 것
칸이 DB와 같은지	칸 이름 5개, 순서, `NULL` 허용 여부가 실제 테이블과 같습니다
기본값	분류를 적지 않으면 `ETC`, `is_system`은 `True`, `created_at`은 채워집니다
같은 분류·같은 이름	두 번 만들면 DB가 거절합니다(`IntegrityError`)
다른 분류·같은 이름	`MOOD`의 `Deep`과 `GENRE`의 `Deep`은 둘 다 들어갑니다(행 2개)

전체 시험 871개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 179 — `SampleTag` 모델과 샘플·태그 관계

항목	내용
파일	`studionx/backend/app/models/tag.py` (`SampleTag` 클래스), `sample.py`·`tag.py`의 `relationship`
작업	S1-07 샘플 라이브러리: ORM 모델 컬럼과 관계 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	178 `Tag` 모델 · (S1-07 다음 함수)

1. 하는 일

`sample_tags` 표는 어느 샘플에 어느 태그가 붙었는지 잇는 표입니다. 이 표를 파이썬 클래스 `SampleTag`로 옮겨 적고, 177·178의 FK를 **파이썬 속성(`relationship`)으로도 오갈 수 있게** 했습니다.

관계가 없을 때	관계가 있을 때
`db.scalars(select(Tag).join(SampleTag).where(SampleTag.sample_id == 3))`	`sample.tags`
`db.scalars(select(Sample).join(SampleTag).where(SampleTag.tag_id == 5))`	`tag.samples`

속성	방향	값
`sample.tags`	샘플 → 태그	`[Tag, …]`
`tag.samples`	태그 → 샘플	`[Sample, …]`

2. 언제 쓰나

쓰는 곳	쓰는 방법
S1-07 태그 추가·삭제 (FR-225)	`sample.tags.append(tag)`로 붙이고, `sample.tags.remove(tag)`로 뗍니다
S1-07 라이브러리 목록 (FR-224)	각 샘플의 `sample.tags`로 붙은 태그를 보여 주고, 태그로 거릅니다
앞선 P1-03의 관계(13번)	`project.members`·`project.tracks`와 같은 방식입니다

```
# 샘플에 태그를 붙일 때
tag = Tag(name="Deep", category=TagCategory.MOOD, is_system=0)
sample.tags.append(tag)
db.add(sample)   # 중간 표(sample_tags) 행은 저장할 때 자동으로 들어갑니다
db.commit()
```

3. 코드

```
class SampleTag(Base):
    __tablename__ = "sample_tags"

    sample_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("samples.sample_id", ondelete="CASCADE"), primary_key=True
```

```
    )
    tag_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("tags.tag_id", ondelete="CASCADE"), primary_key=True
    )
```

관계는 두 클래스에 나누어 적었습니다(서로를 문자열 이름으로 가리킵니다).

```
# sample.py
tags: Mapped[list["Tag"]] = relationship(secondary="sample_tags", back_populates="samples")

# tag.py
samples: Mapped[list["Sample"]] = relationship(secondary="sample_tags", back_populates="tags")
```

SQL과 한 줄씩 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`PRIMARY KEY (sample_id, tag_id)`	두 칸 모두 `primary_key=True`
`FOREIGN KEY (sample_id) REFERENCES samples (sample_id) ON DELETE CASCADE`	`ForeignKey("samples.sample_id", ondelete="CASCADE")`
`FOREIGN KEY (tag_id) REFERENCES tags (tag_id) ON DELETE CASCADE`	`ForeignKey("tags.tag_id", ondelete="CASCADE")`

문법 풀이

식	뜻
복합 기본 키	두 칸을 **합쳐서** 한 행을 가리킵니다. `(3, 5)`는 한 번만 있고, `(3, 6)`과 `(4, 5)`는 따로 있습니다
`ondelete="CASCADE"` (양쪽)	가리키는 샘플이나 태그가 지워지면 **DB가** 이 연결 행도 함께 지웁니다
`secondary="sample_tags"`	다대다 관계에서 거쳐 가는 중간 표의 이름입니다
`back_populates="samples"`	반대쪽 `Tag.samples`와 짝을 이룹니다. 한쪽을 바꾸면 다른 쪽에도 반영됩니다
`TYPE_CHECKING` + 문자열 `"Tag"`	두 파일이 서로를 import하면 순환이 생기므로, 타입 표시용 이름만 문자열로 두고 실행 중에는 불러오지 않습니다

4. 흐름 (샘플을 지울 때)

1. `db.delete(sample)`을 부릅니다. 이때 `samples.deleted_at`만 기록하는 소프트 삭제가 원칙입니다(177번).
2. 관리자가 행을 정말 지우면(`DELETE`), DB가 `ON DELETE CASCADE`로 그 샘플의 `sample_tags` 행을 함께 지웁니다. 태그(`tags`) 행 자체는 남습니다.

5. 왜 이렇게

선택	이유
중간 표에 번호 칸이 없음	`(sample_id, tag_id)`로 이미 한 행이 정해집니다. 같은 샘플에 같은 태그를 두 번 붙일 수 없습니다
양쪽 모두 `ON DELETE CASCADE`	연결 행은 둘 중 하나가 없으면 의미가 없습니다. 소프트 삭제로는 표가 지워지지 않으므로, 이 경우는 관리자가 직접 지울 때만 생깁니다

선택	이유
관계를 양방향(`back_populates`)으로	`sample.tags`로도, `tag.samples`로도 볼 일이 있습니다(태그로 샘플을 거르는 FR-224)
중간 표 클래스를 따로 둠	`relationship(secondary=…)`만으로도 되지만, 시험이나 통계에서 `sample_tags`를 직접 세거나 지울 때가 있어 클래스를 남겼습니다

6. 시험 (`tests/test_tag_model.py` 중 `SampleTag`·관계 부분, 4개)

실제 MySQL의 `sample_tags` 테이블과 비교했습니다. 넣은 행은 시험이 끝나면 되돌립니다.

시험	확인한 것
칸이 DB와 같은지	칸 2개와 FK 2개(둘 다 `ON DELETE CASCADE`)가 실제 테이블과 같습니다
기본 키	`["sample_id", "tag_id"]` 순서입니다
연결과 연쇄 삭제	연결 행을 넣으면 1개가 되고, 샘플 행을 지우면 그 연결 행이 0개가 됩니다(DB의 `CASCADE`)
같은 짝을 두 번	같은 `(sample_id, tag_id)`를 또 넣으면 DB가 거절합니다(`IntegrityError`, 복합 기본 키)

전체 시험 871개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 180 — `read_audio_info()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 업로드 시 길이·샘플레이트·채널 수 추출 (PRD 02 FR-223)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	179 `SampleTag` · 181 파형 미리보기 데이터 (예정)

1. 하는 일

파일을 열어 **헤더만** 읽고 길이(ms)·샘플레이트·채널 수를 뽑습니다. 샘플 내용을 전부 풀어 읽지 않으므로 100MB 파일(FR-222 한도)도 금방입니다.

넣은 것	결과
1초, 44100Hz, 모노 WAV	`AudioInfo(duration_ms=1000, sample_rate=44100, channels=1)`
0.5초, 48000Hz, 스테레오	`500 / 48000 / 2`
10006 표본, 8000Hz	1250.75ms → 반올림하여 `1251`
WAV, MP3, FLAC, OGG, AIFF	다 같은 모양으로 읽힙니다
AU (지원 목록 밖)	이 함수는 **성공**합니다. 형식 목록 검사는 다음 함수가 합니다
없는 파일, 폴더	404 `NOT_FOUND` "Audio file not found."
오디오가 아닌 파일, 빈 파일	415 `UNSUPPORTED_FILE_TYPE` "Use a WAV, MP3, FLAC, OGG or AIFF audio file."
샘플레이트 0 (깨진 헤더)	400 `BAD_REQUEST` "This audio file has an invalid sample rate."

2. 언제 쓰나

쓰는 곳	
샘플 업로드 API (다음 함수)	업로드한 파일의 경로를 넘겨 세 칸을 채웁니다 (PRD 02 §4.2 FR-223)
`samples` 테이블	`duration_ms`, `sample_rate`, `channels` 세 칸 (DB.md §3.2)

파형 미리보기(`waveform_path`)는 FR-223의 나머지 부분이라 다음 함수(181)가 만듭니다.

3. 코드

import (파일 맨 위)

```
from dataclasses import dataclass
from pathlib import Path

import soundfile as sf

from app.core.errors import AppError, ErrorCode
```

```
AUDIO_MESSAGE = "Use a WAV, MP3, FLAC, OGG or AIFF audio file."
```

세 칸을 담는 값 묶음

```
@dataclass(frozen=True)
class AudioInfo:
    """파일 하나에서 읽은 오디오 기본 정보. samples 테이블의 해당 칸에 그대로 채운다.

    duration_ms 는 길이(밀리초), sample_rate 는 초당 표본 수,
    channels 는 채널 수이다 (모노 1, 스테레오 2).
    """

    duration_ms: int
    sample_rate: int
    channels: int
```

함수

```
def read_audio_info(path: str | Path) -> AudioInfo:
    file_path = Path(path)
    if not file_path.is_file():
        raise AppError(ErrorCode.NOT_FOUND, 404, "Audio file not found.")
    try:
        info = sf.info(file_path)
    except sf.SoundFileError as exc:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, AUDIO_MESSAGE) from exc
    if info.samplerate <= 0:
        raise AppError(
            ErrorCode.BAD_REQUEST, 400, "This audio file has an invalid sample rate."
        )
    duration_ms = int(round(info.frames * 1000 / info.samplerate))
    return AudioInfo(
        duration_ms=duration_ms,
        sample_rate=int(info.samplerate),
        channels=int(info.channels),
    )
```

문법 풀이

식	뜻
`@dataclass(frozen=True)`	값이 고정된 값 묶음입니다. `mail_templates.py`와 같은 쓰임입니다
`Path(path)`, `is_file()`	문자열이든 `Path`든 같은 방식으로 파일만 있는지 봅니다
`sf.info(file_path)`	헤더 정보(`frames` 표본 수, `samplerate`, `channels`)만 읽습니다
`sf.SoundFileError`	soundfile이 던지는 오류의 어머니입니다. 형식을 못 알아본 `LibsndfileError`를 포함합니다
`raise … from exc`	원인 오류를 남기고 AppError로 바꿔 던집니다 (Sentry에 두 오류가 함께 남습니다)
`int(round(...))`	소수로 나오는 밀리초를 정수로 반올림합니다

4. 흐름

1. 파일이 있는지 봅니다 (없으면 404).
2. `sf.info()`로 헤더만 읽습니다 (내용은 디코딩하지 않습니다).
3. 형식을 못 읽으면 415, 샘플레이트가 0이면 400으로 바꿔 던집니다.
4. 표본 수(frames) ÷ 샘플레이트 × 1000으로 밀리초를 구해 세 칸을 담아 돌려줍니다.

5. 왜 이렇게

선택	이유
`sf.info()`로 헤더만	길이·샘플레이트·채널 수는 샘플 전체를 메모리로 올리지 않아도 나옵니다. 파일이 커도 열 때 시간이 거의 걸리지 않습니다
형식 목록은 여기서 검사하지 않음	libsndfile은 WAV 빼고 AU·VOC 같은 형식도 엽니다. "허용 목록"과 "libsndfile이 읽을 수 있는가"는 다르므로, 허용 목록 검사는 파일 확장자와 내용을 함께 보는 별도 함수로 둡니다 (다음)
오류를 AppError로 바꿔 던짐	06번(오류 처리기)이 같은 모양으로 응답합니다. 읽지 못한 파일은 업로드 자체가 거부되어, 메타 칸이 NULL로 남지 않습니다
파일이 없으면 404	업로드 흐름에서는 저장한 뒤 다시 읽는 경우입니다. 사라진 샘플 파일은 "없는 것"으로 다루고(FR-2230 파일 관리), 그 형식 문제는 415로 구분합니다
세 칸을 한 값 묶음으로	세 값은 항상 같이 움직입니다. 다음 파형 함수도 같은 파일을 한 번 더 읽기 때문에, 그때도 같은 파일에서 함께 값을 얻습니다

6. 시험 (`tests/test_sample_audio_service.py`에 13개 추가, 전체 884개 통과)

시험용 파일은 임시 폴더(`tmp_path`)에 만들었고, 샘플 값은 0(소리 없음)이라 길이만 확인합니다. 이 함수는 DB를 쓰지 않아 실제 MySQL 시험은 없습니다.

시험	확인한 것
1초 44100Hz 모노	1000 / 44100 / 1
0.5초 48000Hz 스테레오	500 / 48000 / 2
밀리초 반올림	1250.75 → 1251
문자열 경로	`Path`가 아니어도 같은 결과
값 묶음	`AudioInfo`이고 값이 바뀌지 않음 (frozen)
지원 형식 5가지	WAV, MP3, FLAC, OGG, AIFF 모두 읽힘
AU (목록 밖)	이 함수는 성공 — 형식 목록 검사는 다음 함수가 함
없는 파일, 폴더	차례로 404, 404
텍스트 파일, 빈 파일	차례로 415, 415
샘플레이트 0	400 (`sf.info`를 가짜로 바꿔 확인)
파일 내용	함수를 불러도 바뀌지 않음 (읽기 전용)

Backend(FastAPI/Python) 181 — `compute_waveform()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 업로드 시 파형 미리보기 데이터 생성 (PRD 02 FR-223, FR-210)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	180 `read_audio_info()` · 182 파형 파일 저장 (예정)

1. 하는 일

파일을 float32로 읽어 채널을 하나로 섞고, 표본을 `buckets` 칸으로 나눠 칸마다 (최솟값, 최댓값)을 냅니다. 화면은 이 값으로 파형을 그립니다.

넣은 것	결과
1초 44100Hz 모노 (무음)	`peaks` 1000칸, 모두 `(0.0, 0.0)`
일정한 0.5, buckets=10	10칸 모두 `(0.5, 0.5)`
440Hz 사인 0.8, buckets=20	첫 칸은 음수와 양수를 모두 가짐, 모든 값이 −1.0 ~ 1.0
왼쪽 +0.5, 오른쪽 −0.5 (스테레오)	`channels=2`, 모두 `(0.0, 0.0)` (평균이 0)
1000표본, buckets=100	칸을 표본 수인 10개로 줄임 (빈 칸 방지)
float 2.0	1.0으로 자름 (clamp)
0.123456	0.1235로 반올림
없는 파일 / 오디오 아님 / 빈 파일	404 / 415 / 400
buckets 0	`ValueError`

2. 언제 쓰나

업로드 흐름에서 파일 저장 다음에 이어집니다: 파일 저장 → `read_audio_info`(180) → `compute_waveform`(181) → JSON 파일로 저장(182) → `samples.waveform_path`.

쓰는 곳	
라이브러리 패널 미리보기	FR-224
타임라인 클립 파형	FR-210
`samples` 테이블	`waveform_path` (DB.md §3.2, JSON/PNG) — **JSON 피크 배열**로 결정했습니다 (2026-10-15 사용자 결정)

3. 코드

import (파일 맨 위, `numpy` 추가)

```
from dataclasses import dataclass
from pathlib import Path
```

```
import numpy as np
import soundfile as sf

from app.core.errors import AppError, ErrorCode

AUDIO_MESSAGE = "Use a WAV, MP3, FLAC, OGG or AIFF audio file."

#: 파형 미리보기의 기본 칸 수(peaks 길이). 화면 폭(보통 1000px 남짓)에 맞춘다.
DEFAULT_BUCKETS = 1000
```

파형 값 묶음

```
@dataclass(frozen=True)
class Waveform:
    """파형 미리보기 데이터. peaks 는 (최솟값, 최댓값) 쌍의 목록이다 (각 -1.0 ~ 1.0).

    peaks 의 길이가 buckets 이고, 채널이 여럿이면 하나로 섞은(평균) 값을 쓴다.
    """

    sample_rate: int
    channels: int
    peaks: tuple[tuple[float, float], ...]
```

함수

```
def compute_waveform(path: str | Path, buckets: int = DEFAULT_BUCKETS) -> Waveform:
    if buckets < 1:
        raise ValueError("buckets must be 1 or more")
    info = read_audio_info(path)
    try:
        data, _ = sf.read(path, dtype="float32", always_2d=True)
    except sf.SoundFileError as exc:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, AUDIO_MESSAGE) from exc
    total = int(data.shape[0])
    if total == 0:
        raise AppError(ErrorCode.BAD_REQUEST, 400, "This audio file has no samples.")
    mono = data.mean(axis=1)
    count = min(buckets, total)
    edges = np.linspace(0, total, count + 1).astype(np.int64)
    peaks: list[tuple[float, float]] = []
    for i in range(count):
        segment = mono[edges[i] : edges[i + 1]]
        low = float(np.clip(segment.min(), -1.0, 1.0))
        high = float(np.clip(segment.max(), -1.0, 1.0))
        peaks.append((round(low, 4), round(high, 4)))
    return Waveform(sample_rate=info.sample_rate, channels=info.channels, peaks=tuple(peaks))
```

문법 풀이

식	뜻
`sf.read(path, dtype="float32", always_2d=True)`	파일 전체를 읽어 2차원 `(표본, 채널)` 배열로 만듭니다. `always_2d`라서 모노도 2차원입니다
`data.mean(axis=1)`	채널 방향으로 평균을 내어 하나로 섞습니다

식	뜻
`np.linspace(0, total, count + 1).astype(np.int64)`	칸 경계(표본 번호)를 고르게 나눠 정수로 만듭니다
`mono[edges[i] : edges[i + 1]]`	i번째 칸에 든 표본들입니다
`np.clip(x, -1.0, 1.0)`	값을 −1.0 ~ 1.0 밖으로 나가지 않게 자릅니다
`round(x, 4)`	소수 넷째 자리에서 반올림합니다
`tuple(...)`	값이 바뀌지 않는 목록입니다 (frozen 값 묶음과 어울립니다)

4. 흐름

1. `buckets`를 확인합니다 (1 미만이면 `ValueError`).
2. `read_audio_info`(180)로 없는 파일(404)·못 읽는 파일(415)을 먼저 가립니다.
3. `sf.read`로 전체를 float32로 읽습니다.
4. 표본이 하나도 없으면 400입니다.
5. 채널을 평균해 하나로 섞습니다.
6. 표본을 `count`칸(= `min(buckets, 표본 수)`)으로 나눠, 칸마다 최솟값·최댓값을 −1 ~ 1로 자르고 반올림합니다.
7. `sample_rate`, `channels`, `peaks`를 담아 돌려줍니다.

5. 왜 이렇게

선택	이유
JSON 피크 배열 (min/max 쌍)	화면에서 확대·축소할 때 그리는 쪽이 값을 골라 쓰기 쉽습니다. PNG는 크기가 고정되어 확대하면 흐려집니다 (2026-10-15 사용자 결정)
칸마다 min/max	피크를 전부 그리면 화면 폭보다 표본이 많아 무의미합니다. 칸의 최솟값·최댓값만 남기면 파형 모양이 유지됩니다
채널을 평균 (모노)	화면에 한 줄로 그리는 미리보기에 충분합니다. 채널별로 나눠 그리는 것은 필요해지면 더합니다
값 범위를 −1.0 ~ 1.0으로 자름	float WAV는 1.0을 넘을 수 있어 그리기 좌표가 넘칩니다. 그리기용이라 −1 ~ 1로 자릅니다
소수 넷째 자리 반올림	그리기에 그 이상 정밀도는 필요 없고 JSON 파일이 작아집니다
표본보다 칸이 많으면 줄임	빈 칸을 만들면 그릴 때 끊깁니다. 칸 수가 표본 수를 넘지 않게 합니다
`buckets`는 `ValueError`	부르는 쪽(서버)이 정하는 값이라 사용자 입력이 아닙니다. 잘못 부르면 설정 실수이므로 400이 아니라 `ValueError`로 바로 드러냅니다
`read_audio_info`를 먼저 호출	존재·형식 오류(404·415)를 180과 똑같이 맞추려고 재사용합니다. 헤더만 한 번 더 읽어 비용이 거의 없습니다

6. 시험 (`tests/test_sample_audio_service.py`에 15개 추가, 전체 899개 통과)

파일은 임시 폴더(`tmp_path`)에 만들었고, 샘플 값은 직접 넣어 확인합니다.

시험	확인한 것
길이와 정보	`peaks`가 `DEFAULT_BUCKETS`(1000)개이고, 값이 `(float, float)` 쌍이며, `sample_rate=44100`, `channels=1`

시험	확인한 것
무음	모든 칸 `(0.0, 0.0)`
일정한 0.5	10칸 모두 `(0.5, 0.5)`
사인	첫 칸이 음수·양수를 모두 가지고, 모든 값이 −1 ~ 1
스테레오 평균	`channels=2`, 모두 `(0.0, 0.0)`
칸 수	buckets=10 → 10칸, buckets=1 → 1칸
표본보다 큰 buckets	표본 10개 → 칸 10개로 줄임
buckets=1	파일 전체의 최솟값·최댓값 `(-0.5, 0.5)`
자름·반올림	2.0 → `(1.0, 1.0)`, 0.123456 → `(0.1235, 0.1235)`
다른 형식	FLAC도 같은 결과
buckets=0	`ValueError`
없는 파일 / 오디오 아님 / 빈 파일	404 / 415 / 400
값 묶음	`Waveform`이고 값이 바뀌지 않음 (frozen)

Backend(FastAPI/Python) 182 — `write_waveform()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 파형 미리보기 JSON 파일 저장 (PRD 02 FR-223)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	181 `compute_waveform()` · 183 샘플 업로드 (예정)

1. 하는 일

181이 만든 peaks를 JSON 파일로 저장하고, 저장소 기준 상대 경로(`waveforms/{owner_id}/{uuid}.json`)를 돌려줍니다. 이 경로가 `samples.waveform_path`에 들어갑니다.

넣은 것	결과
4칸 peaks, owner 7	`waveforms/7/{32자}.json` 파일이 만들어짐
같은 값을 다시 저장	새 무작위 이름으로 또 만들어짐 (이름이 겹치지 않음)
저장 폴더가 없음	`waveforms/{owner_id}/`까지 만들어짐
저장 위치가 폴더가 아니라 파일	`OSError`가 올라가 500이 됨

파일 내용의 예는 이렇습니다 (빈칸 없이 씁니다).

```
{"version":1,"sample_rate":44100,"channels":1,"peaks":[[0.5,0.5],[0.5,0.5],[0.5,0.5],[0.5,0.5]]}
```

2. 언제 쓰나

업로드 흐름이 이어집니다: 파일 저장 → `read_audio_info`(180) → `compute_waveform`(181) → `write_waveform`(182) → `samples.waveform_path`.

쓰는 곳	
라이브러리 패널 미리보기	FR-224
타임라인 클립 파형	FR-210
화면에 내려보내는 길	`GET /samples/{id}/waveform` (PRD 02 §4.2)

3. 코드

import (파일 맨 위, `json`, `uuid4`, `get_settings` 추가)

```
import json
from dataclasses import dataclass
from pathlib import Path
from uuid import uuid4

import numpy as np
```

```
import soundfile as sf

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode

#: 파형 JSON 을 담는 저장소 폴더와 파일 형식 번호
WAVEFORM_DIR = "waveforms"
WAVEFORM_VERSION = 1
```

함수

```
def write_waveform(waveform: Waveform, owner_id: int) -> str:
    payload = {
        "version": WAVEFORM_VERSION,
        "sample_rate": waveform.sample_rate,
        "channels": waveform.channels,
        "peaks": [list(pair) for pair in waveform.peaks],
    }
    text = json.dumps(payload, separators=(",", ":"))
    relative = f"{WAVEFORM_DIR}/{owner_id}/{uuid4().hex}.json"
    target = get_settings().storage_dir / relative
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(text, encoding="utf-8")
    return relative
```

문법 풀이

식	뜻
`json.dumps(payload, separators=(",", ":"))`	빈칸 없이 씁니다 (파일 크기를 줄입니다)
`uuid4().hex`	32자 무작위 이름입니다. 같은 파일을 다시 처리해도 겹치지 않습니다
`get_settings().storage_dir`	설정의 저장 위치(`studionx/storage`, git에서 제외된 폴더)입니다
`target.parent.mkdir(parents=True, exist_ok=True)`	중간 폴더까지 만들고, 이미 있어도 오류를 내지 않습니다
`write_text(text, encoding="utf-8")`	문자열을 한 번에 씁니다 (한글이 섞여도 utf-8로 고정)

4. 흐름

1. peaks, samplerate, channels를 JSON 문자열로 만듭니다 (빈칸 없이).
2. `waveforms/{owner_id}/{uuid}.json` 상대 경로와 실제 경로를 만듭니다.
3. 폴더를 만들고 파일을 씁니다.
4. 상대 경로를 돌려줍니다.

5. 왜 이렇게

선택	이유
파일로 따로 저장 (DB에 peaks를 안 넣음)	1000칸 peaks는 30KB 남짓이라 DB 행에 넣기엔 큽니다. DB.md가 `waveform_path`(파일 경로)로 정해 둔 것과 맞습니다

선택	이유
무작위 파일 이름	같은 샘플을 다시 처리해도 이름이 겹치지 않습니다. 옛 파형 파일 정리는 부르는 쪽(업로드·삭제)이 맡습니다
상대 경로로 저장	저장소 위치(`storage_dir`)가 바뀌어도 DB 값은 그대로 쓸 수 있습니다 (프로필 사진 38번과 같은 방식)
`version` 칸	나중에 peaks 형식(예: 채널별)이 바뀌면 화면이 `version`을 보고 처리합니다
빈칸 없이 (separators)	peaks 1000쌍이면 공백만으로도 수 KB가 늘어납니다
peaks를 `list`로 변환	JSON은 tuple도 배열로 쓰지만, 넣는 값의 모양을 분명히 하려고 list로 바꿉니다
쓰기 실패는 그대로 `OSError`	서버·디스크 오류라 500으로 가게 둡니다. 06번 오류 처리기가 파일 경로가 새지 않게 일반 문장으로 바꿉니다

6. 시험 (`tests/test_sample_audio_service.py`에 6개 추가, 전체 905개 통과)

파일은 임시 폴더(`tmp_path`)에 만들었고, `storage_dir`를 그 임시 폴더로 바꿔 진짜 `storage/`를 건드리지 않았습니다.

시험	확인한 것
파일 만들기	경로가 `waveforms/7/{32자}.json` 꼴이고 파일이 있음
JSON 내용	`version=1`, `sample_rate`, `channels`, `peaks`가 181의 값과 같음
빈칸 없음	`": "`, `", "`, 줄바꿈이 없음
이름이 겹치지 않음	같은 값을 두 번 저장하면 서로 다른 파일 두 개
폴더 만들기	`waveforms/42/` 폴더가 생김
쓰기 실패	저장 위치가 파일이면 `OSError`가 올라감

Backend(FastAPI/Python) 183 — `check_upload()`

항목	내용
파일	`studionx/backend/app/services/sample_audio_service.py`
작업	S1-07 샘플 라이브러리: 업로드 검사 — 지원 형식·파일 크기(FR-222), 저장 용량(FR-227)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	182 `write_waveform()` · 184 샘플 업로드 저장 (예정)

1. 하는 일

올리려는 파일을 **이름(확장자)과 크기만** 보고 지원 형식·파일 하나 최대 크기·남은 저장 용량을 검사합니다. 문제가 있으면 `AppError`를 던집니다.

넣은 것	결과
`a.wav`, `b.MP3`, `c.Flac`, `d.ogg`, `e.aiff`, `f.aif`	통과
`a.au`, `a.txt`, `a` (확장자 없음), `a.wav.zip`, `a.mp3.exe`	415 `UNSUPPORTED_FILE_TYPE`
100MB (딱 한도)	통과
100MB + 1바이트	413 `FILE_TOO_LARGE` "Audio files must be 100 MB or smaller."
남은 용량 60, 파일 60	통과
남은 용량 60, 파일 61	413 `STORAGE_QUOTA_EXCEEDED` "Not enough storage space. Free up some space and try again."
남은 용량 0 (한도 다 씀)	413 `STORAGE_QUOTA_EXCEEDED`
이름만 `.wav`인 텍스트 파일	여기서는 통과 (내용은 180이 봄)

2. 언제 쓰나

업로드 흐름의 첫 검사입니다: `check_upload`(183) → 파일 저장 → `read_audio_info`(180) → `compute_waveform`(181) → `write_waveform`(182) → `samples` 행.

정하는 값	
파일 하나 최대 크기	설정 `sample_max_bytes` (`.env` `SAMPLE_MAX_BYTES`, 기본 100MB)
지원 형식	상수 `SUPPORTED_AUDIO_EXTENSIONS` (`.wav`, `.mp3`, `.flac`, `.ogg`, `.aiff`, `.aif`)
오류 코드	`STORAGE_QUOTA_EXCEEDED`를 새로 넣었습니다 (`core/errors.py`). 화면이 이 코드를 보고 [용량 정리] 안내를 띄웁니다 (FR-135)

3. 코드

상수 (파일 위쪽)

```
#: 지원하는 오디오 확장자 (FR-222). 대소문자는 가리지 않는다.
SUPPORTED_AUDIO_EXTENSIONS = (".wav", ".mp3", ".flac", ".ogg", ".aiff", ".aif")
```

함수

```
def check_upload(user: User, filename: str, file_size: int) -> None:
    settings = get_settings()
    extension = Path(filename).suffix.lower()
    if extension not in SUPPORTED_AUDIO_EXTENSIONS:
        raise AppError(ErrorCode.UNSUPPORTED_FILE_TYPE, 415, AUDIO_MESSAGE)
    if file_size > settings.sample_max_bytes:
        limit_mb = settings.sample_max_bytes // (1024 * 1024)
        message = f"Audio files must be {limit_mb} MB or smaller."
        raise AppError(ErrorCode.FILE_TOO_LARGE, 413, message)
    remaining_bytes = user.storage_quota_bytes - user.storage_used_bytes
    if file_size > remaining_bytes:
        raise AppError(
            ErrorCode.STORAGE_QUOTA_EXCEEDED,
            413,
            "Not enough storage space. Free up some space and try again.",
        )
```

문법 풀이

식	뜻
`Path(filename).suffix`	마지막 점 뒤의 확장자를 꺼냅니다 (`"a.wav.zip"` → `".zip"`)
`.lower()`	`.WAV` 같은 대문자도 같은 형식으로 봅니다
`settings.sample_max_bytes // (1024 * 1024)`	바이트를 MB로 바꿔 안내 문구에 넣습니다
`user.storage_quota_bytes - user.storage_used_bytes`	남은 저장 용량입니다 (FR-227)

4. 흐름

1. 파일 이름에서 확장자를 떼어 지원 목록에 있는지 봅니다 (없으면 415).
2. 파일 크기가 한도(`sample_max_bytes`)를 넘는지 봅니다 (넘으면 413).
3. 파일 크기가 남은 저장 용량(한도 − 사용량)을 넘는지 봅니다 (넘으면 413, 다른 코드).
4. 아무 문제가 없으면 아무것도 돌려주지 않고 지나갑니다 (통과).

5. 왜 이렇게

선택	이유
확장자만 보고 내용은 180에 맡김	100MB 파일 내용을 다 읽어 검사하면 저장 전에 큰 비용이 듭니다. 형식 이름은 415로 빠르게 거르고, 진짜 오디오인지는 180(헤더)이 확인합니다
413 두 가지를 코드로 구분	"이 파일이 너무 큼"과 "저장 공간이 없음"은 안내가 다릅니다. FR-227은 [용량 정리]로 유도해야 하므로 화면이 `code`로 나눌 수 있게 `STORAGE_QUOTA_EXCEEDED`를 따로 둡니다
남은 용량 = 한도 − 사용량	FR-227·FR-135 그대로입니다. 파일 크기가 남은 용량 이하면 통과합니다 (딱 맞아도 통과)
크기 검사를 용량 검사보다 먼저	파일 자체가 한도를 넘으면 용량과 상관없이 거절합니다. 순서를 고정해 어떤 문구가 나올지 예측할 수 있게 합니다

선택	이유
`.aif`도 허용	AIFF의 확장자는 `.aiff`와 `.aif`가 둘 다 흔합니다
대소문자 무시	`.WAV`·`.MP3`도 같은 형식입니다

6. 시험 (`tests/test_sample_audio_service.py`에 12개 추가, 전체 917개 통과)

검사 대상 값(`storage_quota_bytes`, `storage_used_bytes`)만 채운 `User`를 써서 DB 없이 시험했습니다. 파일 크기 한도는 임시로 작게 바꿔 확인했습니다.

시험	확인한 것
형식 통과	`a.wav`, `b.MP3`, `c.Flac`, `d.ogg`, `e.aiff`, `f.aif`가 통과
형식 거부	`a.au`, `a.txt`, `a`, `a.wav.zip`, `a.mp3.exe`, 빈 이름이 415
파일 크기	딱 한도는 통과, 1바이트 초과는 413 `FILE_TOO_LARGE`이고 문구에 "MB" 포함
남은 용량	남은 60에 60은 통과, 61은 413 `STORAGE_QUOTA_EXCEEDED`
한도 다 씀	사용량 = 한도이면 1바이트도 413
검사 순서	파일 크기 오류와 용량 오류가 겹치면 파일 크기 오류가 먼저
이름만 오디오	`.wav`인 텍스트 파일은 183을 통과하고 180에서 415

Backend(FastAPI/Python) 184 — `save_sample()`

항목	내용
파일	`studionx/backend/app/services/sample_service.py` (새 파일)
작업	S1-07 샘플 라이브러리: 업로드 저장 — 파일 저장, 메타·파형 생성, samples 행, 사용량 증가 (FR-221·131)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	183 `check_upload()` · 185 용량 재계산 (예정)

1. 하는 일

올린 파일을 `storage/samples/{사용자 번호}/{무작위 32자}.{확장자}` 에 저장하고, 같은 트랜잭션에서 `samples` 행을 넣고 `users.storage_used_bytes`를 파일 크기만큼 늘립니다. 완성된 `Sample`을 돌려줍니다.

항목	값
`file_path`	`samples/{owner_id}/{uuid4().hex}{확장자}` (확장자는 소문자)
`waveform_path`	`write_waveform`(182)가 만든 `waveforms/{owner_id}/{uuid}.json`
`name`	올린 파일 이름에서 확장자를 뗀 값 (`kick.wav` → `kick`), 200자까지
`original_filename`	올린 파일 이름 그대로, 255자까지
`mime_type`	검사를 통과한 확장자로 정합니다 (`audio/wav`, `audio/mpeg`, …). 브라우저가 알려 준 값은 믿지 않습니다
`duration_ms` · `sample_rate` · `channels`	`read_audio_info`(180)
`source`	`SampleSource.UPLOAD`
`users.storage_used_bytes`	파일 크기만큼 증가 (FR-131)

2. 언제 쓰나

업로드 흐름의 마지막 단계입니다: `check_upload`(183) → **`save_sample`(184)** → `read_audio_info`(180) → `compute_waveform`(181) → `write_waveform`(182) → `samples` 행. 라우터(`POST /samples`)가 이 함수를 부릅니다.

저장 경로 상수는 `SAMPLE_DIR = "samples"`, 형식별 MIME은 `MIME_TYPES`, 나눠 읽는 크기는 `CHUNK_BYTES`(1MB)입니다.

3. 코드

상수 (파일 위쪽)

```
SAMPLE_DIR = "samples"
MIME_TYPES = {
    ".wav": "audio/wav",
    ".mp3": "audio/mpeg",
    ".flac": "audio/flac",
    ".ogg": "audio/ogg",
```

```
    ".aiff": "audio/aiff",
    ".aif": "audio/aiff",
}
CHUNK_BYTES = 1024 * 1024
```

함수 (뼈대)

```
async def save_sample(db: Session, user: User, file: UploadFile) -> Sample:
    settings = get_settings()
    storage_dir = settings.storage_dir
    original_filename = file.filename or ""
    extension = Path(original_filename).suffix.lower()
    relative = f"{SAMPLE_DIR}/{user.user_id}/{uuid4().hex}{extension}"
    target = storage_dir / relative
    target.parent.mkdir(parents=True, exist_ok=True)

    file_size = 0
    with target.open("wb") as out:
        while True:
            chunk = await file.read(CHUNK_BYTES)
            if not chunk:
                break
            file_size += len(chunk)
            out.write(chunk)
            if file_size > settings.sample_max_bytes:
                break

    waveform_relative: str | None = None
    try:
        check_upload(user, original_filename, file_size)
        info = read_audio_info(target)
        waveform = compute_waveform(target)
        waveform_relative = write_waveform(waveform, owner_id=user.user_id)

        stem = Path(original_filename).stem.strip()
        sample = Sample(
            owner_id=user.user_id,
            name=stem[:200] or "Untitled",
            original_filename=original_filename[:255],
            file_path=relative,
            mime_type=MIME_TYPES[extension],
            file_size=file_size,
            duration_ms=info.duration_ms,
            sample_rate=info.sample_rate,
            channels=info.channels,
            waveform_path=waveform_relative,
            source=SampleSource.UPLOAD,
        )
        db.add(sample)
        user.storage_used_bytes = user.storage_used_bytes + file_size
        db.commit()
    except Exception:
        db.rollback()
        target.unlink(missing_ok=True)
        if waveform_relative:
            (storage_dir / waveform_relative).unlink(missing_ok=True)
```

```
        raise
    db.refresh(sample)
    return sample
```

문법 풀이

식	뜻
`await file.read(CHUNK_BYTES)`	올린 파일에서 1MB씩 읽습니다 (한 번에 100MB를 메모리에 올리지 않습니다)
`with target.open("wb") as out`	저장할 파일을 열어 두고 블록을 벗어날 때 닫습니다
`if file_size > settings.sample_max_bytes: break`	한도를 넘으면 더 읽지 않습니다 (디스크를 다 채우지 않습니다)
`except Exception:`	아래 어느 단계에서든 오류가 나면 파일을 지우고 되돌립니다
`target.unlink(missing_ok=True)`	파일이 이미 없어도 오류 없이 지웁니다
`MIME_TYPES[extension]`	183을 통과한 확장자라 반드시 표에 있습니다

4. 흐름

1. 저장할 경로(`samples/{사용자}/{무작위}.{확장자}`)를 만들고 폴더를 준비합니다.
2. 올린 파일을 1MB씩 나눠 쓰면서 크기를 셉니다. 한도를 넘으면 읽기를 멈춥니다.
3. `check_upload`(183)로 형식·크기·남은 용량을 확인합니다. 걸리면 2에서 쓴 파일을 지우고 그 오류를 그대로 올립니다.
4. `read_audio_info`(180)로 메타를, `compute_waveform`(181)과 `write_waveform`(182)로 파형 파일을 만듭니다. 못 읽는 파일이면 415로 멈추고 쓴 파일을 지웁니다.
5. `samples` 행을 넣고 `user.storage_used_bytes`를 파일 크기만큼 늘려 **한 번의 commit**으로 저장합니다.
6. commit이 실패하면 rollback으로 되돌리고, 남은 파일 둘(샘플·파형)을 지웁니다.

5. 왜 이렇게

선택	이유
파일 저장과 행·사용량을 한 함수에	183이 읽는 사용량을 올리는 곳이 여기뿐입니다. 나누면 `samples` 행만 생기고 사용량이 안 늘어 FR-135(한도 차단)가 동작하지 않습니다
한 번의 commit	행과 사용량이 따로 저장돼 어긋난 상태가 남지 않습니다 (사용자와 합의, 2026-10-15)
오류 시 파일까지 지움	DB는 되돌리지만 디스크 파일은 자동으로 안 지워집니다. 반쪽 상태를 남기지 않으려고 직접 지웁니다
형식 검사를 파일을 쓴 뒤에	100MB를 메모리에 올리지 않으려고 나눠 쓰기 때문입니다. 어차피 걸리면 지우므로 남는 파일이 없습니다
MIME을 확장자로 정함	브라우저가 보내는 `content_type`은 조작될 수 있습니다. 검사를 통과한 확장자를 씁니다 (프로필 사진 38과 같은 방침)
`name`은 파일 이름에서 확장자를 뗌	라이브러리 목록에 보여 줄 이름입니다. 사용자가 따로 정하기 전까지의 기본값입니다
이름·파일 이름 길이 제한	DB 칸(`name` 200자, `original_filename` 255자)에 맞춰 자릅니다

6. 시험 (`tests/test_sample_service.py` 새 파일 12개, 전체 929개 통과)

메모리에서 오디오를 만들어 올리고(`BytesIO`), 저장 위치는 임시 폴더로 바꿉니다. 행과 사용량은 실제 MySQL로 확인합니다.

시험	확인한 것
저장·행 생성	경로 모양, 파일 내용 일치, `name`·`original_filename`·`mime_type`·`file_size`·`duration_ms`·`sample_rate`·`channels`·`source`·`deleted_at`, `sample_id`
사용량 증가	`user.storage_used_bytes`와 DB 값이 파일 크기만큼 늘어남
파형 파일	경로 모양과 JSON 내용(`sample_rate`, `channels`, peaks 1000칸)
FLAC	확장자 `.flac`, `mime_type` `audio/flac`
대소문자	`LOOP.WAV`도 `.wav`, `audio/wav`
확장자 거부	`.txt`면 415, 파일·행·사용량이 남지 않음
못 읽는 파일	이름만 `.wav`인 텍스트는 415(180), 파일·행이 남지 않음
크기 초과	413 `FILE_TOO_LARGE`, 파일·행이 남지 않음
용량 초과	413 `STORAGE_QUOTA_EXCEEDED`, 사용량은 그대로
commit 실패	rollback으로 행이 남지 않고 샘플·파형 파일을 지움
두 번 올리기	파일 이름·파형 이름이 다르고 사용량이 합계만큼 늘어남
긴 파일 이름	`name` 200자, `original_filename` 255자로 잘림

Backend(FastAPI/Python) 185 — `POST /samples`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (새 파일), `app/schemas/sample.py` (새 파일), `app/api/v1/router.py`, `app/services/sample_service.py`(`to_response`)
작업	S1-07 샘플 라이브러리: 업로드 API — `POST /samples` (FR-221·222·227)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	184 `save_sample()` · 186 `GET /samples` 목록 (예정)

1. 하는 일

브라우저가 드롭한 파일 하나를 받아 저장하고, 저장된 샘플 정보를 201로 돌려줍니다. `POST /api/v1/samples`, `multipart/form-data`, 칸 이름 `file`입니다.

요청·결과	내용
성공	201 + `SampleResponse`
로그인 안 함	401
`file` 칸 없음	422 (FastAPI)
지원하지 않는 형식	415 `UNSUPPORTED_FILE_TYPE` (183)
100MB 초과	413 `FILE_TOO_LARGE` (183)
남은 용량 부족	413 `STORAGE_QUOTA_EXCEEDED` (183)

응답 `SampleResponse` 칸: `sample_id`, `name`, `original_filename`, `file_size`, `mime_type`, `duration_ms`, `sample_rate`, `channels`, `bpm`, `music_key`, `source`, `created_at`, `tags`. `tags`는 붙은 태그 목록인데, 186이 목록에서 채웁니다. 업로드 직후에는 아직 태그가 없어 빈 목록입니다.

2. 언제 쓰나

라이브러리 패널·타임라인에 파일을 드롭할 때 부릅니다(FR-221). 라우터는 `save_sample`(184)을 부르고 결과를 `to_response`로 바꿔 돌려주기만 합니다. 검사와 저장은 전부 서비스에 있습니다.

3. 코드

라우터 (`app/api/v1/samples.py`)

```
router = APIRouter()


@router.post(
    "",
    response_model=SampleResponse,
    status_code=status.HTTP_201_CREATED,
    responses={413: {"model": ErrorResponse}, 415: {"model": ErrorResponse}},
)
```

```
async def upload_sample(file: UploadFile, user: CurrentUser, db: Db) -> SampleResponse:
    sample = await sample_service.save_sample(db, user, file)
    return sample_service.to_response(sample)
```

응답 스키마 (`app/schemas/sample.py`)

```
class SampleResponse(BaseModel):
    sample_id: int
    name: str
    original_filename: str
    file_size: int
    mime_type: str
    duration_ms: int | None
    sample_rate: int | None
    channels: int | None
    bpm: Decimal | None
    music_key: str | None
    source: str
    created_at: datetime
    tags: list[TagItem] = []  # 186 이 목록에서 채운다. 업로드 때는 빈 목록
```

응답 변환 (`app/services/sample_service.py`)

```
def to_response(sample: Sample) -> SampleResponse:
    return SampleResponse(
        sample_id=sample.sample_id,
        name=sample.name,
        ...
        source=sample.source,
        created_at=sample.created_at,
    )
```

라우터 등록 (`app/api/v1/router.py`)

```
api_router.include_router(samples.router, prefix="/samples", tags=["samples"])  # S1-07
```

문법 풀이

식	뜻
`UploadFile`	multipart 로 올린 파일 한 개를 받습니다 (프로필 사진 38과 같음)
`CurrentUser`	`deps.py`가 토큰을 검사해 넣어 주는 로그인 사용자입니다
`status.HTTP_201_CREATED`	새 자원을 만들었으므로 200이 아니라 201을 씁니다
`responses={413: …, 415: …}`	이 오류 모양을 문서(OpenAPI)에 표시합니다
`Decimal`	`bpm` 칸은 정확한 소수를 쓰려고 Decimal 로 받습니다 (프로젝트 응답과 같음)

4. 흐름

1. 요청이 오면 `deps.get_current_user`가 먼저 로그인을 확인합니다 (401).
2. `UploadFile`을 꺼냅니다 (`file` 칸이 없으면 422).
3. `sample_service.save_sample`(184)가 저장하고 `Sample`을 돌려줍니다. 형식·크기·용량은 183이 막습니다.

4. `sample_service.to_response`가 응답 모양으로 바꿉니다.
5. 201과 함께 돌려줍니다.

5. 왜 이렇게

선택	이유
라우터는 얇게, 로직은 서비스에	프로젝트·회원 라우터와 같은 방식입니다. 검사·저장을 서비스에 두면 라우터 없이도 시험할 수 있습니다 (184 시험 12개)
칸 이름 `file`	프로필 사진(38)과 같게 맞춰 화면 코드가 한 가지 규칙만 알면 되게 합니다
`to_response`를 서비스에	응답 칸을 하나씩 골라 넣어 DB 칸이 늘어도 응답에 섞이지 않게 합니다 (`to_me_response`와 같은 방식)
형식·크기·용량을 문서에 표시	화면(라이브러리 패널)이 415·413을 코드로 구분해 안내합니다 (FR-227은 정리 유도)
한 번에 한 파일	여러 파일 동시 업로드(FR-226)와 녹음 `source`는 우선순위가 낮아 뒤 작업으로 미룹니다. 지금은 드롭 한 건씩 처리합니다
응답에 파일·파형 주소 없음	그 라우터(`GET /samples/{id}/file`, `/waveform`)를 아직 안 만들었습니다. 만들 때 칸을 더합니다

6. 시험 (`tests/test_sample_routes.py` 새 파일 8개, 전체 937개 통과)

저장 위치는 임시 폴더로 바꾸고, 행·사용량은 실제 MySQL로 확인합니다.

시험	확인한 것
라우트 등록	OpenAPI에 `POST /api/v1/samples`가 있음
업로드 성공	201, 응답 칸 전체, `name`·`mime_type`·`duration_ms`·`source`, DB 행의 소유자·파일 크기
로그인 필요	토큰 없이 401
파일 칸 없음	422
형식 거부	`.txt`면 415 `UNSUPPORTED_FILE_TYPE`
용량 초과	한도까지 쓴 사용자는 413 `STORAGE_QUOTA_EXCEEDED`
크기 초과	한도를 500바이트로 줄이면 413 `FILE_TOO_LARGE`
파일 이름 모양	`samples/{owner_id}/{32자}.wav`

Backend(FastAPI/Python) 186 — `GET /samples`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`list_samples` 추가), `app/schemas/sample.py` (`TagItem`, `SampleListResponse` 추가, `SampleResponse.tags` 추가), `app/services/sample_service.py` (`list_samples`, `_tags_by_sample`, `_escape_like` 추가)
작업	S1-07 샘플 라이브러리: 내 라이브러리 목록 — 이름 검색, 태그 필터, 페이지 (FR-224)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	185 `POST /samples` · 187 `GET /samples/{id}/file` (예정)

1. 하는 일

로그인한 사람의 라이브러리 목록을 돌려줍니다. `GET /api/v1/samples` 입니다.

요청	결과
`GET /samples`	만든 순서로 (나중에 만든 것 먼저), 10개씩
`?query=kic`	이름에 들어간 것만 (대소문자 무시)
`?tag=3`	3번 태그가 붙은 샘플만
`?page=2&size=20`	2페이지, 한 번에 20개 (최대 50)
로그인 안 함	401
`size=1000`	422 (최대 50)

응답은 `{ items, total, page, size }`. 줄 하나는 `SampleResponse` 이고, 붙은 태그가 `tags` 에 들어 있습니다.

이름	내용
`items`	이 페이지 샘플 목록 (만든 순서, 같은 줄은 번호 큰 것 먼저)
`total`	거른 결과 전체 개수 (페이지를 넘겨도 그대로)
`page`, `size`	보낸 값 그대로

2. 언제 쓰나

라이브러리 패널을 열 때 부릅니다(FR-224). 태그 필터(FR-224), 검색, 페이지를 한 함수에서 처리합니다.

3. 코드

라우터 (`app/api/v1/samples.py`)

```
@router.get("", response_model=SampleListResponse)
def list_samples(
    user: CurrentUser,
    db: Db,
    query: Annotated[str, Query(max_length=100)] = "",
    tag: Annotated[int | None, Query(ge=1)] = None,
```

```
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=SAMPLE_PAGE_SIZE_MAX)] = 10,
) -> SampleListResponse:
    return sample_service.list_samples(db, user, query.strip(), tag, page, size)
```

서비스 (`app/services/sample_service.py`)

```
def list_samples(db, user, query, tag, page, size):
    conditions = [Sample.owner_id == user.user_id, Sample.deleted_at.is_(None)]
    if query:
        conditions.append(Sample.name.ilike(f"%{_escape_like(query)}%", escape="\\"))

    statement = select(Sample).where(*conditions)
    if tag is not None:
        statement = statement.join(
            SampleTag, SampleTag.sample_id == Sample.sample_id
        ).where(SampleTag.tag_id == tag)

    total = db.scalar(select(func.count()).select_from(statement.subquery()))
    rows = list(
        db.scalars(
            statement.order_by(Sample.created_at.desc(), Sample.sample_id.desc())
            .offset((page - 1) * size)
            .limit(size)
        ).all()
    )
    tags = _tags_by_sample(db, [sample.sample_id for sample in rows])
    return SampleListResponse(
        items=[to_response(sample, tags.get(sample.sample_id, [])) for sample in rows],
        total=total, page=page, size=size,
    )
```

태그 한 번에 모으기

```
def _tags_by_sample(db, sample_ids):
    rows = db.execute(
        select(SampleTag.sample_id, Tag)
        .join(Tag, Tag.tag_id == SampleTag.tag_id)
        .where(SampleTag.sample_id.in_(sample_ids))
        .order_by(Tag.tag_id)
    ).all()
    grouped: dict[int, list[TagItem]] = {}
    for sample_id, tag in rows:
        grouped.setdefault(sample_id, []).append(
            TagItem(tag_id=tag.tag_id, name=tag.name, category=tag.category)
        )
    return grouped
```

문법 풀이

식	뜻
`.ilike(...)`	대소문자를 가리지 않는 포함 검색입니다
`escape="\\"`	`%`·`_`를 글자 그대로 찾게 하는데, 검색어에 역슬래시가 있으면 먼저 두 배로 바꿉니다 (`_escape_like`)

식	뜻
`statement.subquery()`	거른 결과를 임시 테이블처럼 봐서 줄 수를 셉니다 (프로젝트 목록 58과 같은 방법)
`.offset().limit()`	페이지를 넘기는 자리입니다
`dict.setdefault(key, [])`	샘플 번호별 태그 목록을 채웁니다 (없는 샘플은 빈 목록으로 시작)

4. 흐름

1. 로그인을 확인합니다 (`CurrentUser` → 401).
2. 거를 조건을 모읍니다: 내 것이고, 지우지 않았고, (이름 검색이면) 이름에 글자가 있고, (태그가 있으면) 그 태그가 붙어 있어야 합니다.
3. 거른 조건 그대로 줄 수를 셉니다.
4. 만든 순서로 정렬해 이 페이지 줄만 읽습니다.
5. 이 페이지 샘플들의 태그를 **한 번에** 읽어 샘플별로 묶습니다.
6. `{ items, total, page, size }`로 돌려줍니다.

5. 왜 이렇게

선택	이유
태그 필터를 이름이 아니라 번호로	같은 분류뿐 아니라 분류를 넘겨도 이름이 겹칠 수 있습니다. 화면의 태그 칩은 번호를 이미 알고 있으므로 번호가 안전합니다
태그를 한 번에 모아 읽기	페이지가 50줄이면 줄마다 쿼리하면 50번 더 묻게 됩니다. 프로젝트 목록이 멤버 수를 모아 계산한 것(58)과 같은 방법입니다
태그를 `SampleResponse`에 넣음	라이브러리 패널이 줄마다 태그를 보여 주기 때문입니다. 업로드 응답(185)도 같은 모양을 써야 하므로 한 스키마를 늘렸고, 업로드 때는 빈 목록입니다
이름만 검색	이름 검색(FR-224)만 요구됐습니다. 태그 이름으로도 찾는 일은 태그 필터가 맡습니다
정렬은 만든 순서 하나	라이브러리에서 방금 올린 것이 맨 위에 오는 것이 자연스럽습니다. 이름순·크기순은 쓸 때 추가합니다
페이지 한도 50	프로젝트 목록(`PROJECT_PAGE_SIZE_MAX`)과 같게 맞췄습니다
검색어를 strip	공백만 친 검색어는 전부 보여 줍니다 (프로젝트 목록과 같은 규칙)

6. 시험 (`tests/test_sample_routes.py`에 9개 추가, 전체 946개 통과)

시험	확인한 것
목록 기본	내 것이 나열되고, 다른 사람 것은 안 나옴, 만든 순서, `total`
로그인 필요	토큰 없이 401
이름 검색	부분 일치, 대소문자 무시, `%` 포함 이름
태그 필터	붙은 것만, 없는 태그 번호는 빈 목록
태그 표시	`tags`에 번호 순서로 담김, `category` 값 포함
검색 + 태그	둘 다 거치면 겹치는 것만
페이지	`size=2`에 3개 → 1페이지 2개, `total` 3, 2페이지 1개
지운 샘플	`deleted_at`이 있는 줄은 안 나옴

시험	확인한 것
한도	`size=1000`이면 422

Backend(FastAPI/Python) 187 — `GET /samples/{id}/file`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`read_sample_file` 추가), `app/services/sample_service.py` (`sample_file` 추가)
작업	S1-07 샘플 라이브러리: 오디오 파일 스트리밍 (FR-224 미리듣기)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	186 `GET /samples` · 188 `GET /samples/{id}/waveform` (예정)

1. 하는 일

샘플의 오디오 파일을 그대로 돌려줍니다. `GET /api/v1/samples/{id}/file`입니다. 라이브러리 패널에서 미리듣기(FR-224)와 타임라인 재생에 씁니다.

경우	결과
내 샘플, 파일 있음	200 + 오디오 본문
로그인 안 함	401
없는 번호 · 남의 샘플 · 지운 샘플 · 디스크에 파일 없음	404 (모두 같은 문구)
`If-None-Match`가 지금 ETag와 같음	304 (본문 없음)
`Range: bytes=0-9`	206 + 그 구간만

응답 헤더: `Content-Type`은 DB의 `mime_type`, `ETag`, `Cache-Control: private, no-cache`, `X-Content-Type-Options: nosniff`.

2. 언제 쓰나

브라우저 `<audio src="/api/v1/samples/12/file">`로 소리를 냅니다. ETag로 캐시를 검증하고(PRD 02 §7), Range 요청으로 오디오 탐색을 합니다.

3. 코드

서비스 (`app/services/sample_service.py`)

```
def sample_file(db: Session, user: User, sample_id: int) -> tuple[Sample, Path]:
    sample = get_sample(db, user, sample_id)          # 없음·남의 것·지운 것 → 404
    path = _stored_file(sample.file_path)             # 저장 폴더 밖·파일 없음 → None
    if path is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Sample not found.")
    return sample, path
```

라우터 (`app/api/v1/samples.py`)

```
@router.get(
    "/{sample_id}/file",
```

```
    response_class=FileResponse,
    responses={404: {"model": ErrorResponse}},
)
def read_sample_file(sample_id, request, user, db) -> Response:
    sample, path = sample_service.sample_file(db, user, sample_id)
    etag = f'"{sample.sample_id}-{sample.file_size}-{path.stat().st_mtime_ns}"'
    cache = "private, no-cache"
    if request.headers.get("if-none-match") == etag:
        return Response(status_code=304, headers={"ETag": etag, "Cache-Control": cache})
    return FileResponse(
        path,
        media_type=sample.mime_type,
        headers={"ETag": etag, "Cache-Control": cache, "X-Content-Type-Options": "nosniff"},
    )
```

> 188에서 404 판정을 `get_sample`(소유·삭제)과 `_stored_file`(경로·디스크) 두 도우미로 나누어 파형 서빙과 함께 쓰게 했습니다. `sample_file`은 그 둘을 부르기만 합니다.

문법 풀이

식	뜻
`FileResponse(path, media_type=…)`	파일을 나눠 보내 줍니다. Range 요청도 처리합니다 (Starlette 1.7)
`path.resolve()`	`..` 같은 것을 펴서 실제 경로로 만듭니다
`path.is_relative_to(storage_dir)`	저장 폴더 밖을 가리키지 않는지 확인합니다 (경로 넘어가기 방지)
`path.stat().st_mtime_ns`	파일 수정 시각(나노초)입니다. ETag에 넣어 파일이 바뀌면 값이 달라집니다
`If-None-Match`	브라우저가 "이 ETag 그대로인가?" 하고 물을 때 쓰는 헤더입니다
`status.HTTP_304_NOT_MODIFIED`	안 바뀌었으니 본문을 안 보내겠다는 응답입니다

4. 흐름

1. 로그인을 확인합니다 (401).
2. `sample_file`이 샘플을 찾습니다. 없거나 남의 것이거나 지운 것이면 404.
3. 저장 경로가 저장 폴더 안인지, 디스크에 파일이 있는지 확인합니다. 아니면 404.
4. ETag를 만듭니다 (샘플 번호 + 파일 크기 + 수정 시각).
5. 브라우저가 보낸 `If-None-Match`가 같으면 304로 본문 없이 돌려줍니다.
6. 다르면 `FileResponse`로 파일을 보냅니다.

5. 왜 이렇게

선택	이유
지금은 본인만	PRD 02 §7은 "본인/프로젝트 멤버"지만, 그 샘플을 쓰는 클립이 아직 없습니다. 멤버 열람은 클립을 만들 때 함께 넣습니다
404를 하나로	없는 번호인지, 남의 것인지, 지운 것인지 알려 주지 않습니다. 남의 샘플이 있는지조차 알 수 없게 하려는 것입니다 (프로젝트 `get_membership`과 같은 방침)
ETag로 검증, `no-cache`	샘플 파일은 한 번 올리면 안 바뀌므로, 매번 검증하되 안 바뀌었으면 본문을 안 받게 합니다 (PRD 02 §7 "ETag로 캐시 검증")

선택	이유
Content-Type을 DB 값으로	업로드(185)가 확장자로 정해 둔 값을 그대로 씁니다. 파일 이름을 다시 뜯어보지 않습니다
`nosniff`	브라우저가 내용을 보고 종류를 바꿔 추측하지 않게 합니다 (프로필 사진과 같음)
파일 서빙은 라우터가	경로 찾기·권한은 서비스, 응답 만들기(헤더)는 라우터에 둡니다. 프로필 사진(41)과 같은 나눔입니다

6. 시험 (`tests/test_sample_routes.py`에 7개 추가, 전체 953개 통과)

시험	확인한 것
파일 반환	200, 본문이 올린 바이트와 같음, `Content-Type`·`Cache-Control`·`nosniff`·`ETag`
로그인 필요	토큰 없이 401
남의 샘플	다른 사용자면 404 `NOT_FOUND`
없음/지움	없는 번호 404, 소프트 삭제한 샘플 404
디스크에 없음	행만 있고 파일이 없으면 404
ETag	같은 ETag를 보내면 304, 본문 없음
Range	`bytes=0-9`면 206, 앞 10바이트, `Content-Range` 머리말

Backend(FastAPI/Python) 188 — `GET /samples/{id}/waveform`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`read_sample_waveform` 추가), `app/services/sample_service.py` (`get_sample`·`_stored_file`·`sample_waveform` 추가)
작업	S1-07 샘플 라이브러리: 파형 미리보기 데이터 (FR-223·210)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	187 `GET /samples/{id}/file` · 189 이름 변경·태그·삭제(FR-225) (예정)

1. 하는 일

샘플의 파형 데이터를 돌려줍니다. `GET /api/v1/samples/{id}/waveform`입니다. 라이브러리 패널과 타임라인 클립이 이 값으로 파형을 그립니다(FR-210).

응답은 업로드할 때(182 `write_waveform`) 만들어 둔 JSON 파일을 그대로 보냅니다 (`Content-Type: application/json`).

```
{"version": 1, "sample_rate": 44100, "channels": 1, "peaks": [[-0.32, 0.41], [-0.28, 0.35], ...]}
```

`peaks`의 각 칸은 `[그 구간의 최솟값, 최댓값]`입니다. 칸 수는 최대 1000개(`DEFAULT_BUCKETS`)로, 소리가 길어도 전송량이 일정합니다.

경우	결과
내 샘플, 파형 있음	200 + 파형 JSON
로그인 안 함	401
없는 번호 · 남의 샘플 · 지운 샘플	404 `"Sample not found."`
파형을 아직 안 만든 샘플 · 파일 없음	404 `"Waveform not found."`
`If-None-Match`가 지금 ETag와 같음	304 (본문 없음)

2. 언제 쓰나

```
var res = await fetch("/api/v1/samples/12/waveform");
var wave = await res.json();      // wave.peaks 를 캔버스에 그린다
```

한 번 받은 뒤에는 ETag로 검증해 안 바뀌었으면 본문을 다시 받지 않습니다.

3. 코드

서비스 (`app/services/sample_service.py`)

```
def get_sample(db, user, sample_id) -> Sample:
    sample = db.get(Sample, sample_id)
    if sample is None or sample.deleted_at is not None or sample.owner_id != user.user_id:
```

```
        raise AppError(ErrorCode.NOT_FOUND, 404, "Sample not found.")
    return sample


def _stored_file(relative: str) -> Path | None:
    storage_dir = get_settings().storage_dir.resolve()
    path = (storage_dir / relative).resolve()
    if not path.is_relative_to(storage_dir) or not path.is_file():
        return None
    return path


def sample_waveform(db, user, sample_id) -> tuple[Sample, Path]:
    sample = get_sample(db, user, sample_id)
    if sample.waveform_path is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Waveform not found.")
    path = _stored_file(sample.waveform_path)
    if path is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Waveform not found.")
    return sample, path
```

라우터 (`app/api/v1/samples.py`)

```
@router.get(
    "/{sample_id}/waveform",
    response_class=FileResponse,
    responses={404: {"model": ErrorResponse}},
)
def read_sample_waveform(sample_id, request, user, db) -> Response:
    sample, path = sample_service.sample_waveform(db, user, sample_id)
    etag = f'"{sample.sample_id}-waveform-{path.stat().st_mtime_ns}"'
    cache = "private, no-cache"
    if request.headers.get("if-none-match") == etag:
        return Response(status_code=304, headers={"ETag": etag, "Cache-Control": cache})
    return FileResponse(
        path,
        media_type="application/json",
        headers={"ETag": etag, "Cache-Control": cache, "X-Content-Type-Options": "nosniff"},
    )
```

문법 풀이

식	뜻
`sample.waveform_path is None`	파형을 아직 안 만든 샘플입니다. 이때는 파일을 찾지 않고 바로 404입니다
`f'"{…}-waveform-{…}"'`	파형 ETag에는 `-waveform-`을 붙여 오디오 ETag(187)와 겹치지 않게 합니다
`media_type="application/json"`	파형은 항상 JSON이므로 DB 값을 쓰지 않고 고정합니다

4. 흐름

1. `get_sample`이 내 샘플인지 확인합니다. 아니면 404 "Sample not found.".
2. `waveform_path`가 비어 있으면 404 "Waveform not found.".
3. 저장 폴더 안의 실제 파일인지 확인합니다. 아니면 404 "Waveform not found.".

4. ETag(샘플 번호 + `waveform` + 수정 시각)를 만들어 `If-None-Match`와 비교합니다.
5. 같으면 304, 다르면 JSON 파일을 보냅니다.

5. 왜 이렇게

선택	이유
만든 파일을 그대로 보냄	파형은 올릴 때 182가 한 번 만들고 안 바뀝니다. 요청마다 다시 계산하면 낭비입니다
JSON 피크 배열(사용자 결정 2026-10-15)	PNG·피크 서버 대신 JSON으로 보내 브라우저가 캔버스에 그립니다. 확대·색 바꾸기가 자유롭고 서버가 가볍습니다
`Sample not found.`와 `Waveform not found.` 구분	샘플은 내 것이 확실한데 파형만 없는 경우라, 화면이 "파형 없음"을 구분해 표시할 수 있게 합니다 (남의 샘플은 여전히 "Sample not found.")
404 판정을 도우미로 분리	187·188이 소유·경로 검사를 똑같이 필요로 합니다. `get_sample`·`_stored_file`로 묶어 규칙이 한 곳에 있게 했습니다
파형도 ETag·캐시	한 번 받으면 거의 안 바뀌므로 본문 전송을 아낍니다. 오디오(187)와 같은 방식으로 맞춰 두었습니다

6. 시험 (`tests/test_sample_routes.py`에 6개 추가, 전체 959개 통과)

시험	확인한 것
파형 반환	200, `application/json`, `version`·`sample_rate`·`channels`, `peaks` 1000칸, 각 칸이 2개
로그인 필요	토큰 없이 401
남의 샘플	다른 사용자면 404 `NOT_FOUND`
파형 없음	`waveform_path`가 빈 샘플이면 404 `"Waveform not found."`
디스크에 없음	경로만 있고 파일이 없으면 404
ETag	같은 ETag를 보내면 304, 본문 없음

> 187 시험도 `make_sample`이 돌려주는 `sample.sample_id`를 쓰도록 함께 정리했습니다(결과는 그대로, 전체 959개 통과).

Backend(FastAPI/Python) 189 — `PATCH /samples/{id}`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`update_sample` 라우트), `app/services/sample_service.py` (`update_sample` 추가), `app/schemas/sample.py` (`SampleName`·`SampleUpdateRequest` 추가)
작업	S1-07 샘플 라이브러리: 이름 변경 (FR-225)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	188 `GET /samples/{id}/waveform` · 190 태그 설정 `PUT /samples/{id}/tags` (예정)

1. 하는 일

샘플 이름을 바꿉니다. `PATCH /api/v1/samples/{id}`에 `{"name": "새 이름"}`을 보냅니다. 라이브러리 패널에서 이름을 고쳐 보낼 때 씁니다(FR-225).

경우	결과
내 샘플, 이름이 올바름	200 + 바뀐 줄 (SampleResponse)
로그인 안 함	401
없는 번호 · 남의 샘플 · 지운 샘플	404 `"Sample not found."`
이름이 null · 공백뿐 · 200자 초과	422

이름 앞뒤 공백은 지운 뒤 저장합니다.

2. 언제 쓰나

```
await fetch("/api/v1/samples/12", {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "새 이름" }),
});
```

돌려받은 줄로 목록의 그 항목을 그대로 바꿔 그리면 됩니다(태그도 함께 옵니다).

3. 코드

스키마 (`app/schemas/sample.py`)

```
SAMPLE_NAME_MAX = 200  # samples.name 칸 길이(DB VARCHAR(200))와 같음

SampleName = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=SAMPLE_NAME_MAX)
]


class SampleUpdateRequest(BaseModel):
    name: SampleName | None = None
```

```
    @field_validator("name")
    @classmethod
    def _name_not_null(cls, value: str | None) -> str:
        if value is None:
            raise ValueError("name cannot be empty")
        return value
```

서비스 (`app/services/sample_service.py`)

```
def update_sample(db, user, sample_id, data) -> SampleResponse:
    sample = get_sample(db, user, sample_id)
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(sample, field, value)
    db.commit()
    db.refresh(sample)
    tags = _tags_by_sample(db, [sample.sample_id]).get(sample.sample_id, [])
    return to_response(sample, tags)
```

라우터 (`app/api/v1/samples.py`)

```
@router.patch(
    "/{sample_id}",
    response_model=SampleResponse,
    responses={404: {"model": ErrorResponse}},
)
def update_sample(sample_id, data, user, db) -> SampleResponse:
    return sample_service.update_sample(db, user, sample_id, data)
```

문법 풀이

식	뜻
`SampleName | None = None`	이름을 안 보내도 되게 합니다(PATCH라 한 칸만 보내는 경우 대비)
`StringConstraints(strip_whitespace=True, …)`	검사 전에 앞뒤 공백을 지웁니다. 지운 뒤 1자 이상, 200자 이하
`data.model_dump(exclude_unset=True)`	보낸 칸만 꺼냅니다. 안 보낸 칸은 빠져서 건드리지 않습니다
`setattr(sample, field, value)`	꺼낸 칸 이름 그대로 객체에 넣습니다
`_tags_by_sample(...)`	붙어 있는 태그를 모아 응답에 담습니다(목록 한 줄과 같은 모양)

4. 흐름

1. `get_sample`이 내 샘플인지 확인합니다. 아니면 404.
2. 보낸 이름을 앞뒤 공백을 지워 검사합니다. 빈 값·200자 초과면 422(스키마가 먼저 막음).
3. 이름을 바꾸고 `db.commit()` 합니다.
4. 붙어 있는 태그를 모아 `SampleResponse`로 돌려줍니다.

5. 왜 이렇게

선택	이유
PATCH(부분 수정)	이름만 바꾸는 요청이라 PUT(전체 교체)보다 맞습니다. 보낸 칸만 바꾸는 규칙은 `PATCH /projects`(P1-13)와 같습니다
이름을 스키마에서 검사	빈 값·길이 규칙을 라우터마다 되풀이하지 않게 한 곳에 둡니다. 프로젝트 제목(`ProjectTitle`)과 같은 방식입니다
null 은 422	이름은 비울 수 없습니다. 아예 안 보내면 아무것도 안 바꾸고, null 을 보내면 422로 알려 줍니다
응답에 태그 포함	목록 한 줄과 모양을 맞춥니다. 화면이 응답으로 그 항목만 통째로 바꿔 그릴 수 있습니다
응답 모양은 SampleResponse	업로드·목록과 같은 한 줄 모양을 써서 화면 쪽 처리(그리기)가 하나로 끝납니다

6. 시험 (`tests/test_sample_routes.py`에 9개 추가, 전체 968개 통과)

시험	확인한 것
이름 변경	200, 앞뒤 공백 지워짐, 응답 칸(ITEM_KEYS) 그대로, DB에 반영
태그 유지	태그가 붙은 샘플을 바꿔도 응답에 그 태그가 남음
로그인 필요	토큰 없이 401
남의 샘플	다른 사용자면 404, 이름 안 바뀜
없음/지움	없는 번호 404, 소프트 삭제한 샘플 404
잘못된 이름(4가지)	null · 빈 칸 · 공백뿐 · 201자 → 422, 이름 안 바뀜

Backend(FastAPI/Python) 190 — `PUT /samples/{id}/tags`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`set_sample_tags` 라우트), `app/services/sample_service.py` (`set_sample_tags` 추가), `app/schemas/sample.py` (`TagId`·`SampleTagsRequest` 추가)
작업	S1-07 샘플 라이브러리: 태그 설정 (FR-225)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	189 `PATCH /samples/{id}` · 191 `DELETE /samples/{id}` 소프트 삭제 (예정)

1. 하는 일

샘플에 붙은 태그를 바꿉니다. `PUT /api/v1/samples/{id}/tags`에 `{"tag_ids": [1, 2, 3]}`을 보냅니다. 라이브러리 패널에서 태그를 고쳐 보낼 때 씁니다(FR-225).

보낸 목록이 그 샘플의 태그 전부가 됩니다. 지금 붙어 있던 것은 떼고 새로 붙입니다. 빈 목록(`[]`)은 태그를 모두 떼는 뜻입니다.

경우	결과
내 샘플, 태그 번호가 모두 있음	200 + 바뀐 줄 (SampleResponse)
로그인 안 함	401
없는 번호 · 남의 샘플 · 지운 샘플	404 `"Sample not found."`
목록에 없는 태그 번호가 하나라도 있음	422 + `details.tag_ids` (아무것도 안 바꿈)

2. 언제 쓰나

```
await fetch("/api/v1/samples/12/tags", {
  method: "PUT",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ tag_ids: [3, 7] }),
});
```

돌려받은 줄로 목록의 그 항목을 그대로 바꿔 그립니다(새 태그가 함께 옵니다).

3. 코드

스키마 (`app/schemas/sample.py`)

```
TagId = Annotated[int, Field(ge=1)]


class SampleTagsRequest(BaseModel):
    tag_ids: list[TagId] = []
```

서비스 (`app/services/sample_service.py`)

```
def set_sample_tags(db, user, sample_id, tag_ids) -> SampleResponse:
    sample = get_sample(db, user, sample_id)
    wanted = list(dict.fromkeys(tag_ids))  # 보낸 차례를 지키며 중복을 뺀다
    if wanted:
        found = set(db.scalars(select(Tag.tag_id).where(Tag.tag_id.in_(wanted))).all())
        missing = [tag_id for tag_id in wanted if tag_id not in found]
        if missing:
            raise AppError(
                ErrorCode.VALIDATION_ERROR, 422, "Unknown tag.",
                details={"tag_ids": missing},
            )
    db.execute(delete(SampleTag).where(SampleTag.sample_id == sample.sample_id))
    db.add_all(SampleTag(sample_id=sample.sample_id, tag_id=tag_id) for tag_id in wanted)
    db.commit()
    tags = _tags_by_sample(db, [sample.sample_id]).get(sample.sample_id, [])
    return to_response(sample, tags)
```

라우터 (`app/api/v1/samples.py`)

```
@router.put(
    "/{sample_id}/tags",
    response_model=SampleResponse,
    responses={404: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def set_sample_tags(sample_id, data, user, db) -> SampleResponse:
    return sample_service.set_sample_tags(db, user, sample_id, data.tag_ids)
```

문법 풀이

식	뜻
`TagId = Annotated[int, Field(ge=1)]`	태그 번호는 1 이상입니다. 0·음수는 422
`list(dict.fromkeys(tag_ids))`	보낸 차례를 지키면서 같은 번호를 하나로 냅니다(dict는 차례를 기억함)
`Tag.tag_id.in_(wanted)`	보낸 번호들 중 실제로 있는 것만 한 번에 찾습니다
`missing = [...]`	보낸 번호 중 없는 것의 목록입니다. 비어 있지 않으면 422
`delete(SampleTag).where(...)`	그 샘플의 기존 연결을 통째로 지웁니다(샘플 행은 그대로)
`db.add_all(...)`	새 연결을 한꺼번에 넣습니다. 지우기와 넣기가 한 트랜잭션입니다

4. 흐름

1. `get_sample`이 내 샘플인지 확인합니다. 아니면 404.
2. 보낸 번호에서 중복을 뺍니다.
3. 보낸 번호가 모두 실제 태그인지 확인합니다. 하나라도 없으면 422(그대로 두고 끝).
4. 그 샘플의 기존 연결을 지웁니다.
5. 보낸 번호로 새 연결을 넣고 `db.commit()` 합니다.
6. 붙은 태그를 모아 `SampleResponse`로 돌려줍니다.

5. 왜 이렇게

선택	이유
PUT(전체 교체)	화면은 "이 태그들로 해라"라고 한 번에 보냅니다. 하나씩 더하고 빼는 방식보다 화면·서버 어느 쪽이든 상태가 어긋나지 않습니다
없는 번호는 422	조용히 무시하면 오타로 태그가 빠져도 성공으로 보여 혼란이 생깁니다. 하나라도 없으면 아무것도 안 바꾸고 422로 알려 줍니다
빈 목록 허용	태그를 모두 떼는 것도 정상 동작입니다. 목록이 비었다고 막지 않습니다
중복 제거	`sample_tags`는 (sample_id, tag_id) 복합 기본 키라 같은 번호를 두 번 넣으면 DB 오류가 납니다. 서비스에서 하나로 냅니다
응답은 바뀐 줄	목록 한 줄과 모양을 맞춥니다(189와 같음). 화면이 그 항목만 다시 그립니다
189(이름)·190(태그) 분리	이름은 PATCH, 태그는 별도 경로 PUT으로 나눠 화면이 필요한 것만 보냅니다

6. 시험 (`tests/test_sample_routes.py`에 8개 추가, 전체 976개 통과)

시험	확인한 것
태그 설정	200, 응답에 새 태그, DB에 연결 생김
교체	기존 태그를 떼고 보낸 것만 남음
비우기	`[]`이면 태그가 모두 사라짐
중복	같은 번호를 두 번 보내도 한 번만 붙음
없는 번호	422, `details.tag_ids`에 그 번호, 기존 태그 그대로
잘못된 번호	`0`은 422
로그인 필요	토큰 없이 401
남의 샘플	다른 사용자면 404

Backend(FastAPI/Python) 191 — `DELETE /samples/{id}`

항목	내용
파일	`studionx/backend/app/api/v1/samples.py` (`delete_sample` 라우트), `app/services/sample_service.py` (`delete_sample` 추가)
작업	S1-07 샘플 라이브러리: 소프트 삭제 · 저장 용량 반환 (FR-225·135)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	190 `PUT /samples/{id}/tags` · 192 `GET /tags` 태그 목록 (예정)

1. 하는 일

샘플을 지웁니다. `DELETE /api/v1/samples/{id}` 입니다(FR-225). 행은 남기고 `deleted_at` 만 채우는 **소프트 삭제**이며, 지운 만큼 저장 용량을 돌려줍니다(FR-135).

경우	결과
내 샘플	204 (본문 없음)
로그인 안 함	401
없는 번호 · 남의 샘플 · 이미 지운 샘플	404 `"Sample not found."`

지운 샘플은 목록(186)·오디오(187)·파형(188)·이름 변경(189)·태그(190)에서 모두 404로 빠집니다.

2. 언제 쓰나

```
await fetch("/api/v1/samples/12", { method: "DELETE" });
```

지운 뒤에는 목록을 다시 불러 용량 표시와 목록을 갱신합니다.

3. 코드

서비스 (`app/services/sample_service.py`)

```
def delete_sample(db: Session, user: User, sample_id: int) -> None:
    sample = get_sample(db, user, sample_id)
    sample.deleted_at = datetime.now(UTC).replace(tzinfo=None)  # DB 는 UTC 를 시간대 없이 저장
    user.storage_used_bytes = max(0, user.storage_used_bytes - sample.file_size)
    db.commit()
```

라우터 (`app/api/v1/samples.py`)

```
@router.delete(
    "/{sample_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={404: {"model": ErrorResponse}},
)
```

```
def delete_sample(sample_id: int, user: CurrentUser, db: Db) -> None:
    sample_service.delete_sample(db, user, sample_id)
```

문법 풀이

식	뜻
`datetime.now(UTC).replace(tzinfo=None)`	지금 시각을 UTC로 구해 시간대 표시를 뗍니다. DB가 시간대 없이(naive) UTC를 저장하기 때문입니다
`max(0, user.storage_used_bytes - sample.file_size)`	용량에서 파일 크기를 뺍니다. 혹시라도 더 커서 음수가 되지 않게 0 아래로 내려가지 않게 합니다
`status.HTTP_204_NO_CONTENT`	"지웠고 돌려줄 내용은 없음"을 뜻하는 상태 코드입니다
`-> None`	본문을 돌려주지 않습니다 (204라 본문이 없음)

4. 흐름

1. `get_sample`이 내 샘플인지 확인합니다. 없거나 이미 지운 것이면 404.
2. `deleted_at`에 지금 시각을 넣습니다.
3. `storage_used_bytes`에서 그 파일 크기를 뺍니다.
4. `db.commit()` 하고 204로 끝냅니다.

5. 왜 이렇게

선택	이유
소프트 삭제	행을 실제로 지우지 않아 실수로 지운 것을 되살릴 여지를 남깁니다. 프로젝트 삭제(P1-13)와 같은 방식입니다
파일은 남김	지우면 되돌릴 수 없습니다. 실제 파일 정리는 뒤 정리 작업(FR-137)이 오래된 것부터 맡습니다
용량은 바로 반환	FR-135는 "파일을 지우면 용량이 돌아온다"입니다. 사용자는 지운 즉시 다시 올릴 수 있어야 합니다
두 번 지우면 404	`get_sample`이 이미 지운 것을 빼므로 용량이 두 번 빠지지 않습니다. 지우기는 한 번만 됩니다
204(본문 없음)	돌려줄 내용이 없습니다. 화면은 성공 여부만 알면 되고 목록을 다시 불러 갱신합니다

6. 시험 (`tests/test_sample_routes.py`에 6개 추가, 전체 982개 통과)

시험	확인한 것
삭제	204, 본문 없음, `deleted_at` 채워짐, 사용량이 파일 크기만큼 줄어 0이 됨
목록·재생에서 빠짐	지운 뒤 목록에서 사라지고, 오디오·파형은 404
두 번 삭제	두 번째는 404, 용량을 또 빼지 않음
로그인 필요	토큰 없이 401
남의 샘플	다른 사용자면 404, 안 지워짐
없는 번호	404

Backend(FastAPI/Python) 192 — `GET /tags`

항목	내용
파일	`studionx/backend/app/api/v1/tags.py` (신설), `app/services/tag_service.py` (신설), `app/schemas/tag.py` (신설), `app/api/v1/router.py` (`/tags` 등록), `app/schemas/sample.py` (TagItem 을 tag.py 에서 가져오도록 정리)
작업	S1-07 샘플 라이브러리: 태그 목록 (FR-224·225)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	191 `DELETE /samples/{id}` · 193 (예정) — S1-07 백엔드 마무리 점검

1. 하는 일

태그 목록을 돌려줍니다. `GET /api/v1/tags` 입니다. 라이브러리 패널의 태그 필터를 채울 때 씁니다(FR-224).

```
{"items": [
  {"tag_id": 1, "name": "Hip-Hop", "category": "GENRE"},
  {"tag_id": 5, "name": "Kick", "category": "INSTRUMENT"},
  {"tag_id": 9, "name": "Chill", "category": "MOOD"}
]}
```

경우	결과
로그인함	200 + `{ items: [...] }`
로그인 안 함	401

정렬은 **분류 순(GENRE → INSTRUMENT → MOOD → ETC), 같은 분류 안에서는 이름순**입니다.

2. 언제 쓰나

```
var res = await fetch("/api/v1/tags");
var tags = (await res.json()).items;   // category 로 나눠 필터를 그린다
```

필터에서 고른 태그의 `tag_id` 를 그대로 `GET /samples?tag=`(186)와 `PUT /samples/{id}/tags`(190)에 보냅니다.

3. 코드

스키마 (`app/schemas/tag.py`)

```
class TagItem(BaseModel):
    tag_id: int
    name: str
    category: str


class TagListResponse(BaseModel):
    items: list[TagItem]
```

서비스 (`app/services/tag_service.py`)

```
def list_tags(db: Session) -> TagListResponse:
    rows = db.scalars(select(Tag).order_by(Tag.category, Tag.name)).all()
    return TagListResponse(
        items=[
            TagItem(tag_id=tag.tag_id, name=tag.name, category=tag.category)
            for tag in rows
        ]
    )
```

라우터 (`app/api/v1/tags.py`)

```
router = APIRouter()


@router.get("", response_model=TagListResponse)
def list_tags(user: CurrentUser, db: Db) -> TagListResponse:
    return tag_service.list_tags(db)
```

`router.py`에 `api_router.include_router(tags.router, prefix="/tags", tags=["tags"])`로 등록합니다.

문법 풀이

식	뜻
`select(Tag).order_by(Tag.category, Tag.name)`	분류 먼저, 그 안에서 이름순으로 정렬합니다
`db.scalars(...).all()`	그 조건에 맞는 Tag 행을 모두 가져옵니다
`TagItem(tag_id=…, name=…, category=…)`	ORM 행에서 필요한 칸만 골라 응답 항목으로 옮깁니다
`api_router.include_router(..., prefix="/tags")`	`/api/v1/tags` 아래에 이 라우터를 붙입니다

4. 흐름

1. 로그인을 확인합니다 (401).
2. 분류 순, 이름순으로 태그를 모두 읽습니다.
3. 필요한 칸만 골라 `{ items: [...] }`로 돌려줍니다.

5. 왜 이렇게

선택	이유
평평한 목록 + `category`	분류별로 묶어 주지 않고 각 항목에 `category`를 담습니다. 화면이 나누어 그리면 되고, 태그가 늘어도 응답 모양이 바뀌지 않습니다 (사용자 결정 2026-10-15)
분류 순 정렬	MySQL `tags.category`는 ENUM이라 정의된 차례(GENRE → INSTRUMENT → MOOD → ETC)로 정렬됩니다. 화면이 따로 정렬하지 않아도 됩니다
읽기만	태그는 기본 제공(is_system) 목록입니다. 만들기·고치기는 화면 기획이 정해지면 추가합니다
`schemas/tag.py` 분리	태그는 샘플과 다른 자원이고, 한 줄 모양(TagItem)을 샘플 응답과 함께 씁니다. 중복 정의를 피하려고 tag.py에 두고 sample.py 가 가져다 씁니다

선택	이유
`/tags` 별도 라우터	태그는 샘플에 매이지 않은 목록입니다. `/samples` 아래 두지 않고 `/tags`로 둡니다

6. 시험 (`tests/test_tag_routes.py` 신설, 4개, 전체 986개 통과)

시험	확인한 것
라우트 등록	OpenAPI 에 `GET /api/v1/tags` 가 있음
목록	200, 각 항목 칸(`tag_id`·`name`·`category`), 분류 순 · 이름 순
빈 목록	태그가 없어도 200, `items` 가 목록
로그인 필요	토큰 없이 401